Self-Driving Dynamic Routing and BGP over Secure Transport

Project ID:

CPP3-77

College / University:

College of Engineering, Trivandrum

APJ Abdul Kalam Technological University

Team Members:

Agnij T Dev
Angel Roy
Devanarayanan H
Gopika Sreekumar
Rohan A Jacob

Mentor:

Muthiah Sivappirakasam
Principal Engineer, Hewlett Packard Enterprise

Contents

Part I

Self-Driving Dynamic Routing Using OSPF and BGP in the BIRD Routing Stack

Chapter 1

Topology and Network Design

1.1 Overview

The project is implemented as a Docker-based virtual routing testbed that models a small enterprise network connected to an external upstream network. The topology demonstrates self-healing dynamic routing using the BIRD routing stack with OSPF, BGP, BFD, ECMP, Graceful Restart, and LLGR.

The network contains nine router containers and three host containers. Each router runs BIRD 2.0.8 and participates in one or more routing protocols. The host containers generate end-to-end traffic and verify the effect of routing failures on actual packet forwarding.

The design intentionally separates the network into three major parts:

1. An internal enterprise routing domain using OSPF.
2. A WAN/upstream edge using BGP.
3. End hosts attached to different parts of the topology for traffic validation.

This separation makes it possible to test both internal failures and external WAN edge failures in a controlled way.

1.2 Topology Diagram

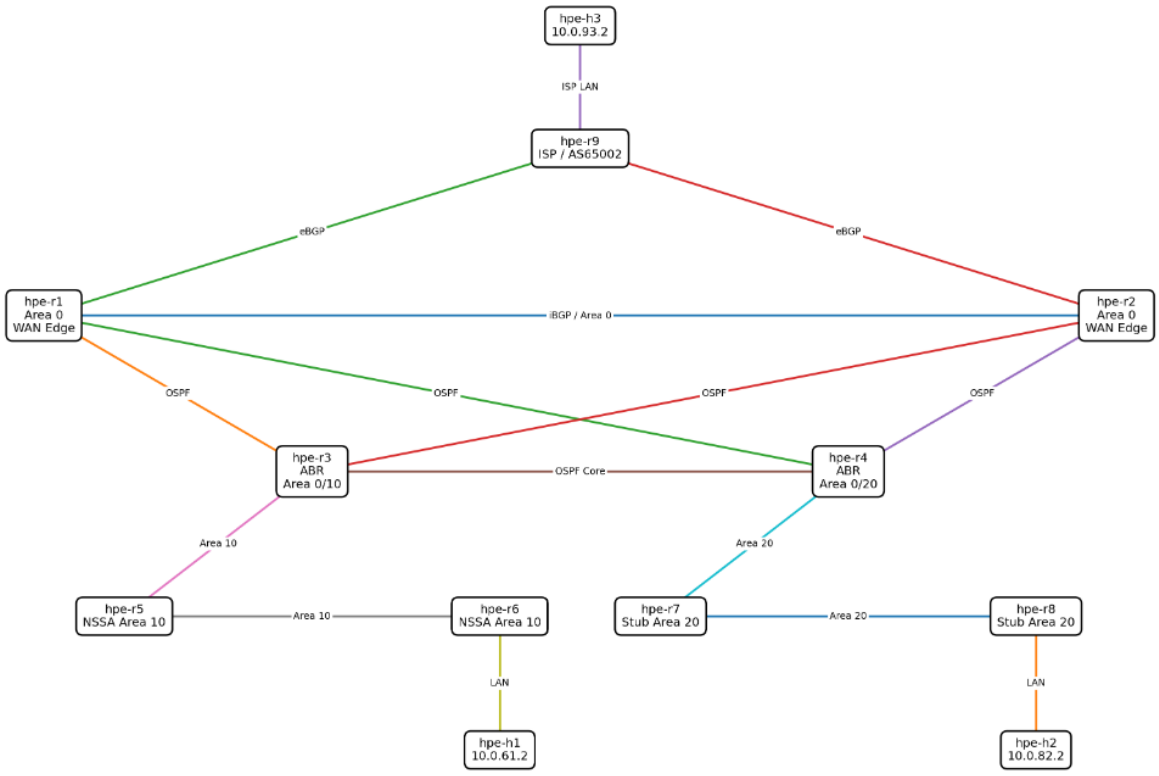


Figure 1.1: Docker-based enterprise routing topology using BIRD, OSPF, BGP, BFD, ECMP, GR and LLGR

1.3 Router and Host Roles

Node	Role in the Topology
`hpe-r1`	Enterprise WAN edge router and OSPF backbone router
`hpe-r2`	Enterprise WAN edge router and OSPF backbone router
`hpe-r3`	Area Border Router connecting OSPF Area 0 and Area 10
`hpe-r4`	Area Border Router connecting OSPF Area 0 and Area 20
`hpe-r5`	Internal router in OSPF NSSA Area 10
`hpe-r6`	Edge router for host `hpe-h1` in Area 10
`hpe-r7`	Internal router in OSPF Stub Area 20
`hpe-r8`	Edge router for host `hpe-h2` in Area 20
`hpe-r9`	External upstream / ISP router in a separate BGP AS
`hpe-h1`	End host behind `hpe-r6`
`hpe-h2`	End host behind `hpe-r8`
`hpe-h3`	End host behind upstream router `hpe-r9`

Routers `hpe-r1`, `hpe-r2`, `hpe-r3`, and `hpe-r4` form the core and backbone. Routers `hpe-r5` and `hpe-r6` represent one internal area, while `hpe-r7` and `hpe-r8` represent another. Router `hpe-r9` represents an external upstream network.

1.4 IP Addressing and Host Placement

Table 1.2: Host IP addressing and placement

Host	IP Address	Connected Router	Default Gateway	Purpose
hpe-h1	10.0.61.2/24	hpe-r6	10.0.61.3	Internal enterprise host in Area 10
hpe-h2	10.0.82.2/24	hpe-r8	10.0.82.3	Internal enterprise host in Area 20
hpe-h3	10.0.93.2/24	hpe-r9	10.0.93.3	External/upstream host

This placement validates multiple traffic paths: internal-to-internal (hpe-h1 ↔ hpe-h2), internal-to-external (hpe-h1 ↔ hpe-h3), and stub-area-to-external (hpe-h2 ↔ hpe-h3).

1.5 OSPF Design

Table 1.3: OSPF area design

OSPF Area	Routers	Area Type	Purpose
Area 0	hpe-r1, hpe-r2, hpe-r3, hpe-r4	Backbone	Core enterprise routing
Area 10	hpe-r3, hpe-r5, hpe-r6	NSSA	Internal area with controlled external route handling
Area 20	hpe-r4, hpe-r7, hpe-r8	Stub	Internal area with reduced external route flooding

Area 0 acts as the OSPF backbone. Routers hpe-r3 and hpe-r4 act as Area Border Routers. Area 10 is configured as NSSA to support Type-7 to Type-5 translation testing. Area 20 is a stub area to reduce unnecessary external route flooding.

1.6 BGP Design

Table 1.4: BGP AS assignment and roles

Router	AS Number	BGP Role
hpe-r1	AS65001	Enterprise edge router
hpe-r2	AS65001	Enterprise edge router
hpe-r9	AS65002	External upstream / ISP router

The important WAN edge paths are:

Table 1.5: WAN edge BGP paths

Path	Purpose
`hpe-r1` → `hpe-r9`	One enterprise-to-upstream BGP path
`hpe-r2` → `hpe-r9`	Second enterprise-to-upstream BGP path
`hpe-r1` ↔ `hpe-r2`	Internal backup/alternate BGP path

1.7 BFD Design

BFD is used to provide fast failure detection. In this project, BFD is attached to both OSPF and BGP. The primary BFD session for WAN edge failure detection is:

```
hpe-r2 <-> hpe-r9    Peer: 10.0.29.3
BFD interval: 0.100 s    BFD timeout: 0.500 s
```

The single-hop BFD target for this project is detection time under 300 ms.

1.8 ECMP Design

The topology includes equal-cost paths inside the OSPF domain. ECMP allows a router to keep multiple equal-cost next-hops for the same destination, providing path redundancy inside the enterprise network. This supports direct ECMP interface-down failover, silent ECMP blackhole failure, and comparison of recovery with and without BFD.

1.9 Control Plane and Data Plane Separation

A key design point is the difference between control plane and data plane. The control plane is handled by BIRD (routing protocols). The data plane is handled by the Linux kernel (packet forwarding). Because of this, the project checks both:

```
birdc show route <prefix> all    # BIRD control-plane route
ip route get <destination-ip>    # Linux kernel forwarding
   decision
```

Routing convergence may appear at different times in these two layers.

1.10 Summary

The topology is a multi-router, multi-protocol routing testbed using Docker containers and BIRD. It evaluates routing convergence at five layers: failure detection, routing protocol reaction, BIRD route-table update, Linux kernel forwarding change, and end-to-end traffic recovery.

Chapter 2

Tools, Environment and Technology Stack

The project was implemented using a Linux-based virtual routing lab. Docker containers emulate routers and hosts, allowing the complete topology to run on a single system without physical hardware. The main routing software is **BIRD 2.0.8**.

Category	**Tools / Technologies**	**Purpose**
Virtualization	Docker	Emulate routers and hosts as containers
Operating System	Ubuntu Linux	Host environment
Routing Stack	BIRD 2.0.8	Run OSPF, BGP, BFD and route control logic
Routing Protocols	OSPF, BGP	Internal and external routing
Fast Detection	BFD	Sub-second failure detection
Redundancy	ECMP	Multiple equal-cost paths and failover
State Preservation	GR, LLGR	BGP restart and stale-route behaviour
Network Tools	`ip`, `ip route get`, `ip monitor route`	Interface control and forwarding observation
Failure Injection	`ip link`, `tc/netem`	Simulate link failures and silent packet drops
Traffic Testing	`ping`, `ping -D`	Reachability, packet loss and outage measurement
Packet Capture	`tcpdump`	Capturing protocol packets for evidence
Automation	Bash scripts	Running experiments and restoring the lab
Data Analysis	Python scripts	Parsing logs and summarizing results

Category	**Tools**	**Purpose**
Documentation	LaTeX	Preparing the final technical report

The lab was organized under `~/Documents/HPE_bird`. All configurations, scripts, measurement logs, result CSV files, screenshots and report files were stored there, making experiments reproducible and verifiable.

Chapter 3

Baseline Validation

3.1 Connectivity and Protocol Validation

Before running any failure experiment, the complete network was validated to confirm that the topology, routing protocols, and host connectivity were working correctly.

The baseline validation checked:

- All router and host containers were running.
- BIRD was running on all router containers.
- OSPF and BGP protocol sessions were active.
- BFD sessions were up where configured.
- End-to-end host connectivity worked with 0% packet loss.

Table 3.1: Baseline connectivity validation results

Test Flow	Source	Destination	Expected Result
h1 to h2	`hpe-h1`	10.0.82.2	0% packet loss
h2 to h1	`hpe-h2`	10.0.61.2	0% packet loss
h1 to h3	`hpe-h1`	10.0.93.2	0% packet loss
h3 to h1	`hpe-h3`	10.0.61.2	0% packet loss
h2 to h3	`hpe-h2`	10.0.93.2	0% packet loss
h3 to h2	`hpe-h3`	10.0.82.2	0% packet loss

```
============================================================
8. Final CSV
============================================================
timestamp,test,source,target,tx,rx,loss_percent,status
20260627_164217,h1_to_h2,hpe-h1,10.0.82.2,5,5,0,pass
20260627_164217,h2_to_h1,hpe-h2,10.0.61.2,5,5,0,pass
20260627_164217,h1_to_h3,hpe-h1,10.0.93.2,5,5,0,pass
20260627_164217,h3_to_h1,hpe-h3,10.0.61.2,5,5,0,pass
20260627_164217,h2_to_h3,hpe-h2,10.0.93.2,5,5,0,pass
20260627_164217,h3_to_h2,hpe-h3,10.0.82.2,5,5,0,pass

Evidence saved to: evidence/final_validation/final_project_validation_20260627_164217.txt
CSV saved to: results/final_validation/final_project_validation_20260627_164217.csv
Latest CSV: results/final_validation/final_project_validation.csv
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$
```

Figure 3.1: Baseline validation result before failure experiments

3.2 Configuration Snapshot

After baseline validation, router configurations were saved as a reproducible reference.

```
Snapshot directory: configs/baseline/<timestamp>/
Latest snapshot:   configs/baseline/latest/
```

Chapter 4

BFD WAN Edge Failure Detection

4.1 Objective

The objective was to validate fast failure detection on the WAN edge using BFD. The failed WAN edge path was `hpe-r2` → `hpe-r9`, with destination `10.0.93.2` (host `hpe-h3` behind `hpe-r9`).

The alternate path expected after failure:

```
hpe-r2 -> hpe-r1 -> hpe-r9
```

4.2 Configuration Details

BFD was enabled on the WAN edge path between `hpe-r2` and `hpe-r9`. The observed BFD session showed:

```
10.0.29.3     eth2     Up     interval 0.100     timeout 0.500
```

```
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ docker exec hpe-r2 ip route get 10.0.93.2
docker exec hpe-r2 birdc show bfd sessions
docker exec hpe-r2 birdc show protocols r9
10.0.93.2 via 10.0.29.3 dev eth2 src 10.0.29.2 uid 0
    cache
BIRD 2.0.8 ready.
bfd1:
IP address                Interface  State      Since         Interval  Timeout
10.0.29.3                 eth2       Up         10:06:14.517    0.100    0.500
BIRD 2.0.8 ready.
Name       Proto      Table      State  Since         Info
r9         BGP        ---        up     10:06:57.808  Established
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$
```

Figure 4.1: BFD WAN precheck showing active direct route via 10.0.29.3, BFD session Up, and BGP r9 Established before failure injection

4.3 Methodology

The failure was created by bringing down the active WAN edge interface on `hpe-r2`. The failure timestamp was recorded as T_0. After failure, multiple layers were monitored: BFD

state, BGP state, BIRD route table, Linux forwarding, kernel route event, traffic impact, and packet capture evidence.

The experiment was repeated across **14 valid runs** to avoid depending on a single result.

4.4 Results

Table 4.1: BFD WAN edge repeated-run summary (14 runs)

Metric	Avg	Median	Min	Max	StdDev
BFD detection time (ms)	72.07	75.00	32.00	121.00	26.57
BGP non-established (ms)	62.21	64.50	0.00	124.00	33.04
BIRD route changed (ms)	71.21	76.50	18.00	124.00	31.17
Linux forwarding changed (ms)	62.93	66.00	1.00	128.00	34.31
Kernel route event (ms)	8.07	8.00	5.00	12.00	2.09
Packet loss (%)	0.01	0.00	0.00	0.13	0.04

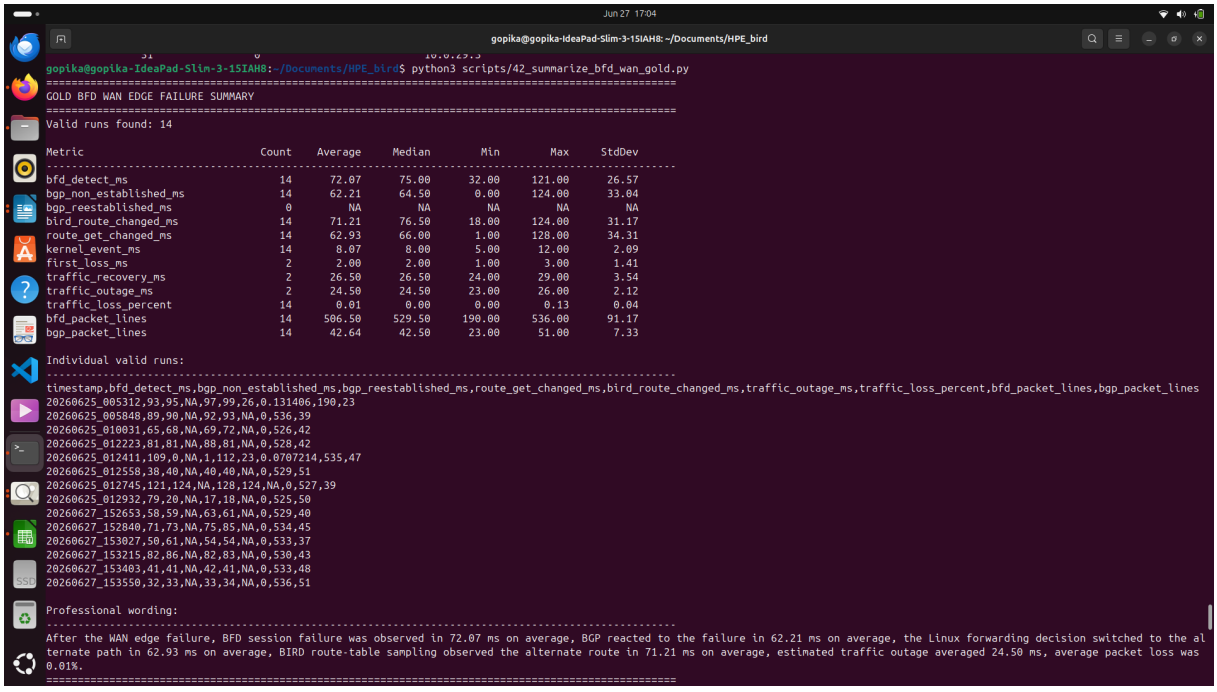


Figure 4.2: Terminal output showing BFD WAN summary statistics across 14 runs

4.5 BFD Flap Behaviour Validation

A repeated-flap test validated BFD stability using `tc netem loss 100%` (silent failure without interface shutdown). Five flap cycles were executed.

Table 4.2: BFD flap test results

Metric	Result
Total cycles	5
Passed cycles	5/5
Average Down detection	656.60 ms
Average recovery time	467.60 ms

```
BFD Repeated Flap Behaviour Test
Session: hpe-r2 <-> hpe-r9
Failure method: tc netem loss 100%

Cycle   hpe-r2 before   hpe-r9 before   Down detect ms   Recovery ms   hpe-r2 after   hp
e-r9 after    Result
1       Up              Up             665              473           Up             Up
              pass
2       Up              Up             657              445           Up             Up
              pass
3       Up              Up             670              474           Up             Up
              pass
4       Up              Up             656              480           Up             Up
              pass
5       Up              Up             635              466           Up             Up
              pass

Cycles passed: 5/5
Average BFD Down detection: 656.60 ms
Average BFD recovery: 467.60 ms
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$
```

Figure 4.3: Repeated BFD flap validation – all five Up → Down → Up cycles passed

4.6 Traffic Impact

Only 2 of 14 runs showed detectable traffic outage (average 24.50 ms when it occurred). Overall average packet loss across all 14 runs was **0.01%**.

4.7 Interpretation and Conclusion

The project target for single-hop BFD failure detection was below 300 ms.

- Average BFD detection time: **72.07 ms**
- Maximum BFD detection time: **121.00 ms**
- All 14 runs were well within the 300 ms target.

BFD detected the WAN edge failure quickly. BGP and route forwarding reacted within a similar time. Average packet loss was negligible at 0.01%, confirming near-continuous reachability during the failure window.

Chapter 5

OSPF Core Link Failure Recovery

5.1 Objective

The objective was to measure how the internal OSPF domain reacts when an active core link fails. The experiment focused on the core path toward target `10.0.82.2` (`10.0.82.0/24`).

```
Failure router: hpe-r3    Failed interface: eth1
Old next-hop:   10.0.34.3
New next-hop:   10.0.13.2  (after failure)
```

5.2 Methodology

The failure was injected by bringing down the active interface on `hpe-r3`. The exact failure timestamp was recorded as T_0. The experiment monitored BIRD route table, Linux forwarding decision, kernel route event, OSPF protocol state, traffic impact, and packet capture. It was repeated across **22 valid runs**.

5.3 Results

Table 5.1: OSPF core link failure repeated-run summary (22 runs)

Metric	Average Result	Interpretation
Linux forwarding decision changed	85.32 ms	Kernel selected alternate next-hop quickly
BIRD route-table update observed	634.95 ms	BIRD showed new route after convergence
Traffic outage	678.50 ms	Estimated duration of traffic disruption
Packet loss	4.45%	Average packet loss during failure recovery

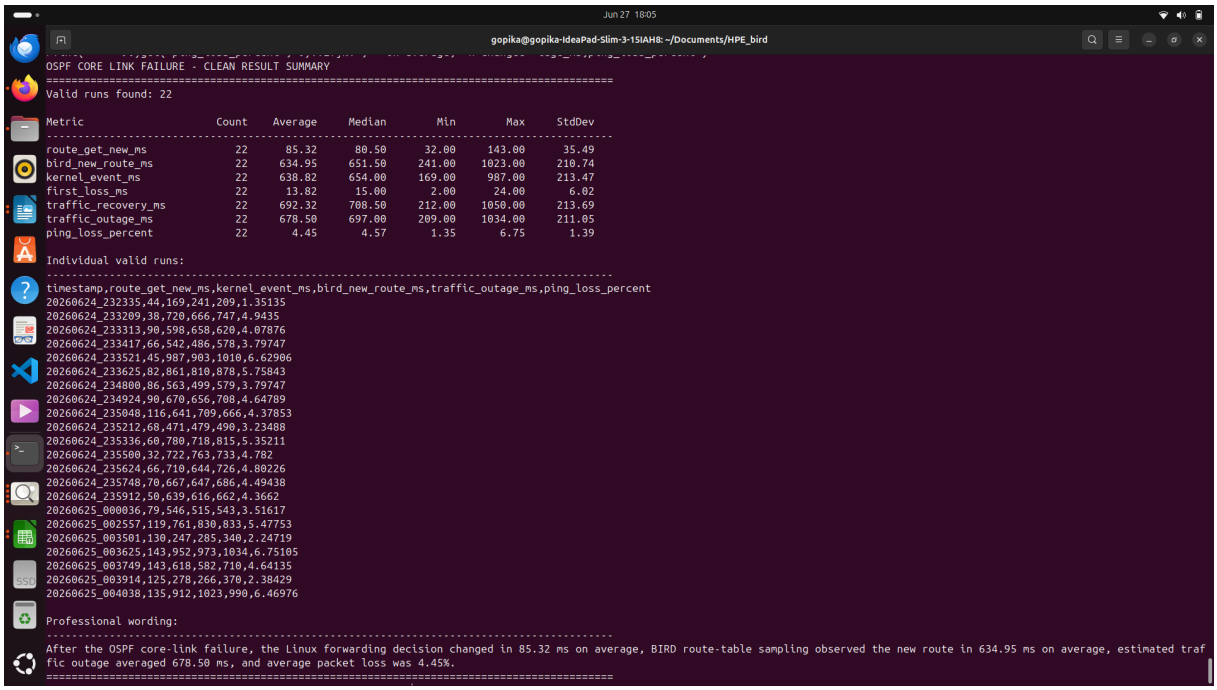


Figure 5.1: OSPF core link failure summary showing repeated-run convergence results

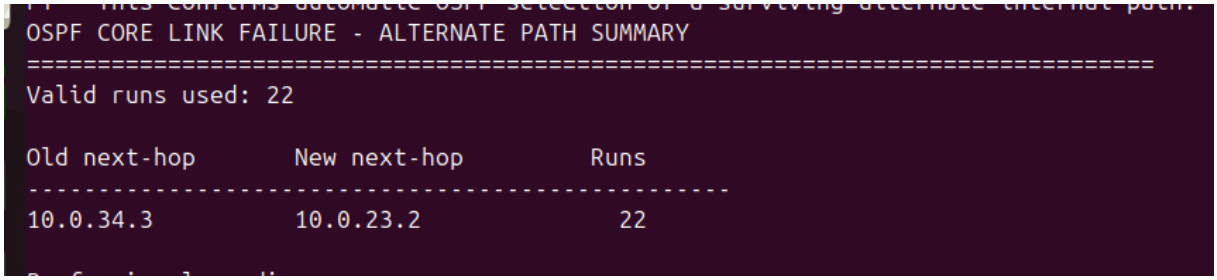


Figure 5.2: Alternate next-hop selected after OSPF core link failure – route movement from 10.0.34.3 to 10.0.23.2 across 22 valid runs

5.4 Conclusion

The OSPF core link failure experiment confirmed automatic internal route recovery. The Linux forwarding decision changed in **85.32 ms** on average. Traffic outage averaged **678.50 ms** with **4.45%** packet loss. After reconvergence, the network returned to full connectivity through the alternate path.

Chapter 6

OSPF ECMP Failover and Re-Hash

6.1 Objective

The objective was to validate OSPF Equal-Cost Multi-Path failover and observe ECMP re-hash to surviving next-hops. The experiment ran in two modes: OSPF ECMP without BFD and OSPF ECMP with BFD.

```
Target:         10.0.24.2 (10.0.24.0/24)
Traffic:        hpe-h1 -> 10.0.24.2
Failed:         hpe-r3 eth2,  next-hop 10.0.34.3  (hpe-r3 -> hpe-
   r4)
Surviving:      next-hop 10.0.23.2                 (hpe-r3 -> hpe
   -r2)
```

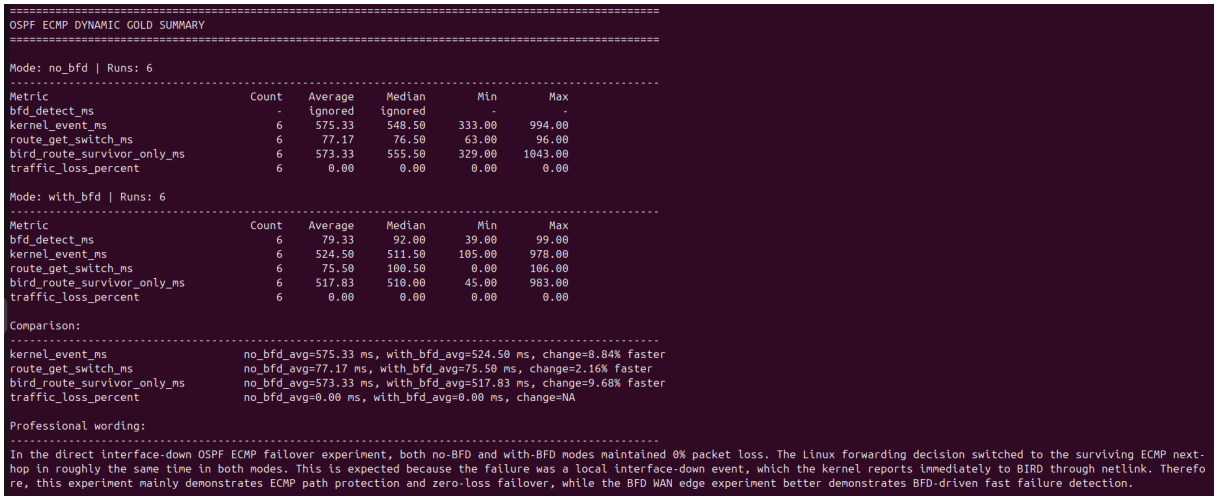
```
OSPF ECMP GOLD RUN - NEXT-HOP TRANSITION
============================================================

Mode: no_bfd
Run directory: measurement/runs/20260625_232200_ospf_ecmp_dynamic_gold_no_bfd
------------------------------------------------------------
Metadata:
timestamp=20260625_232200
test_name=ospf_ecmp_dynamic_gold_no_bfd
fail_router=hpe-r3
target_ip=10.0.24.2
failed_next_hop=10.0.34.3
survivor_next_hop=10.0.23.2
fail_iface=eth2
traffic_src=hpe-h1
traffic_dst=10.0.24.2

Summary:
- Kernel route event observed: 520 ms
- ip route get switched to survivor: 87 ms
- BIRD route became survivor-only: 500 ms
- ip route get missing/unreachable observed: NA ms
- First estimated packet loss: NA ms
- Traffic recovery after loss: NA ms
- Estimated traffic outage duration: NA ms
- Packet loss percent: 0%
- route-get switch line: 1782409939880|state=survivor_nh|10.0.24.2 via 10.0.23.2 dev eth1 src 10.0.23.3 uid 0 cache

- BIRD survivor-only line: 1782409940293|state=survivor_only|BIRD 2.0.8 ready. Table master4: 10.0.24.0/24 unicast
[ospf1 17:52:20.309] * I (150/20) [4.4.4.4] via 10.0.23.2 on eth1 Type: OSPF univ OSPF.metric1: 20 OSPF.router_id:
4.4.4.4
```

Figure 6.1: OSPF ECMP gold run metadata and summary – failover from 10.0.34.3 to 10.0.23.2

```
============================================================

Mode: with_bfd
Run directory: measurement/runs/20260625_230521_ospf_ecmp_dynamic_gold_with_bfd
------------------------------------------------------------
Metadata:
timestamp=20260625_230521
test_name=ospf_ecmp_dynamic_gold_with_bfd
fail_router=hpe-r3
target_ip=10.0.24.2
failed_next_hop=10.0.34.3
survivor_next_hop=10.0.23.2
fail_iface=eth2
traffic_src=hpe-h1
traffic_dst=10.0.24.2

Summary:
- Kernel route event observed: 667 ms
- ip route get switched to survivor: 0 ms
- BIRD route became survivor-only: 683 ms
- ip route get missing/unreachable observed: NA ms
- First estimated packet loss: NA ms
- Traffic recovery after loss: NA ms
- Estimated traffic outage duration: NA ms
- Packet loss percent: 0%
- route-get switch line: 1782408940793|state=survivor_nh|10.0.24.2 via 10.0.23.2 dev eth1 src 10.0.23.3 uid 0 cache

- BIRD survivor-only line: 1782408941476|state=survivor_only|BIRD 2.0.8 ready. Table master4: 10.0.24.0/24 unicast
[ospf1 17:35:41.457] * I (150/20) [4.4.4.4] via 10.0.23.2 on eth1 Type: OSPF univ OSPF.metric1: 20 OSPF.router_id:
4.4.4.4

============================================================
```

Figure 6.2: OSPF ECMP gold run – 0% packet loss during failover

6.2 Results



```
================================================================================================
OSPF ECMP DYNAMIC GOLD SUMMARY
================================================================================================

Mode: no_bfd | Runs: 6
------------------------------------------------------------------------------------------------
Metric                          Count    Average     Median        Min        Max
bfd_detect_ms                       -    ignored    ignored          -          -
kernel_event_ms                     6     575.33     548.50     333.00     994.00
route_get_switch_ms                 6      77.17      76.50      63.00      96.00
bird_route_survivor_only_ms         6     573.33     555.50     329.00    1043.00
traffic_loss_percent                6       0.00       0.00       0.00       0.00

Mode: with_bfd | Runs: 6
------------------------------------------------------------------------------------------------
Metric                          Count    Average     Median        Min        Max
bfd_detect_ms                       6      79.33      92.00      39.00      99.00
kernel_event_ms                     6     524.50     511.50     105.00     978.00
route_get_switch_ms                 6      75.50     100.50       0.00     106.00
bird_route_survivor_only_ms         6     517.83     510.00      45.00     983.00
traffic_loss_percent                6       0.00       0.00       0.00       0.00

Comparison:
------------------------------------------------------------------------------------------------
kernel_event_ms                 no_bfd_avg=575.33 ms, with_bfd_avg=524.50 ms, change=8.84% faster
route_get_switch_ms             no_bfd_avg=77.17 ms, with_bfd_avg=75.50 ms, change=2.16% faster
bird_route_survivor_only_ms     no_bfd_avg=573.33 ms, with_bfd_avg=517.83 ms, change=9.68% faster
traffic_loss_percent            no_bfd_avg=0.00 ms, with_bfd_avg=0.00 ms, change=NA

Professional wording:
------------------------------------------------------------------------------------------------
In the direct interface-down OSPF ECMP failover experiment, both no-BFD and with-BFD modes maintained 0% packet loss. The Linux forwarding decision switched to the surviving ECMP next-
hop in roughly the same time in both modes. This is expected because the failure was a local interface-down event, which the kernel reports immediately to BIRD through netlink. Therefo
re, this experiment mainly demonstrates ECMP path protection and zero-loss failover, while the BFD WAN edge experiment better demonstrates BFD-driven fast failure detection.
```

Figure 6.3: OSPF ECMP dynamic gold summary comparing no-BFD and with-BFD failover results

Across six no-BFD and six with-BFD runs:

- Linux forwarding decision changed within approximately **75–77 ms**.
- Packet loss remained **0%** in both modes.

6.3 ECMP Re-Hash Validation

40 synthetic flow lookups were sampled using `ip route get` before and after failure.

Table 6.1: ECMP re-hash before and after failure

State	Next-hop	Flow Count
Before failure	10.0.34.3 (failed)	17
Before failure	10.0.23.2 (survivor)	23
After failure	10.0.34.3 (failed)	0
After failure	10.0.23.2 (survivor)	40

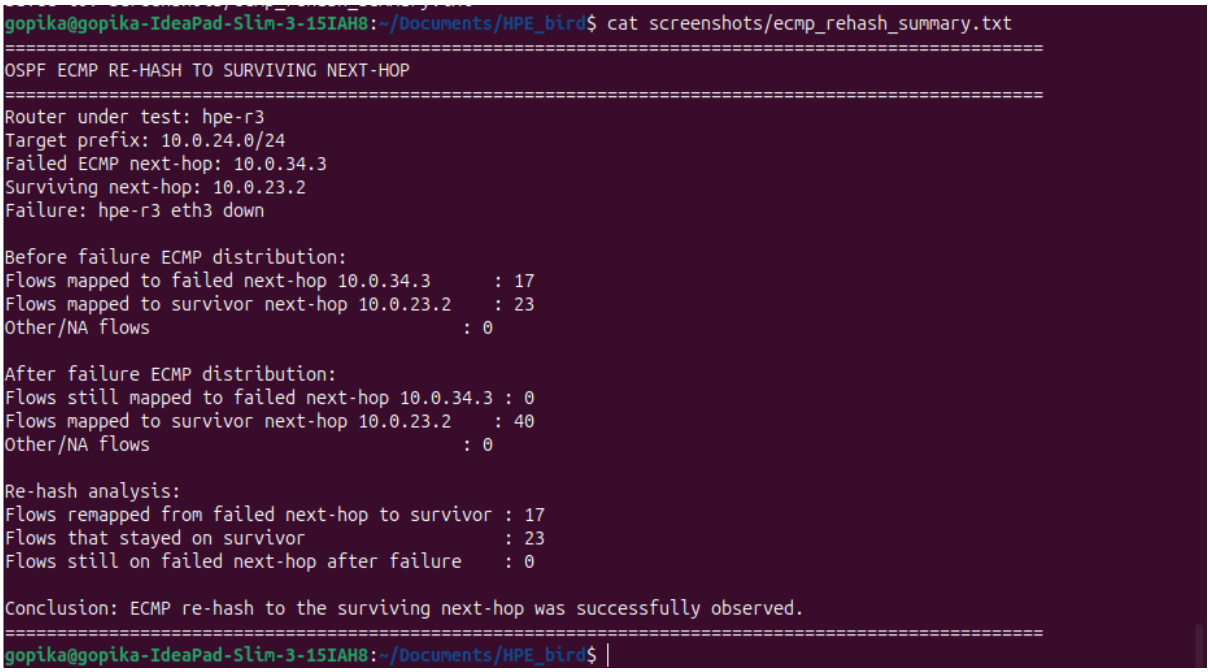


Figure 6.4: ECMP re-hash to the surviving next-hop – all 40 sampled flows remapped after failure

6.4 Conclusion

OSPF ECMP failover worked correctly in both modes with 0% packet loss and ∼75 ms forwarding switch time. The re-hash test confirmed that all 17 flows originally mapped to the failed next-hop were recalculated and moved to the surviving next-hop after failure.

Chapter 7

OSPF ECMP Silent Blackhole Failure

7.1 Objective

The objective was to measure OSPF ECMP failover during a **silent blackhole failure** – where the interface remains physically and administratively up, but traffic is silently dropped. This is harder to detect than a normal interface-down event and is the clearest proof of BFD's value in the project.

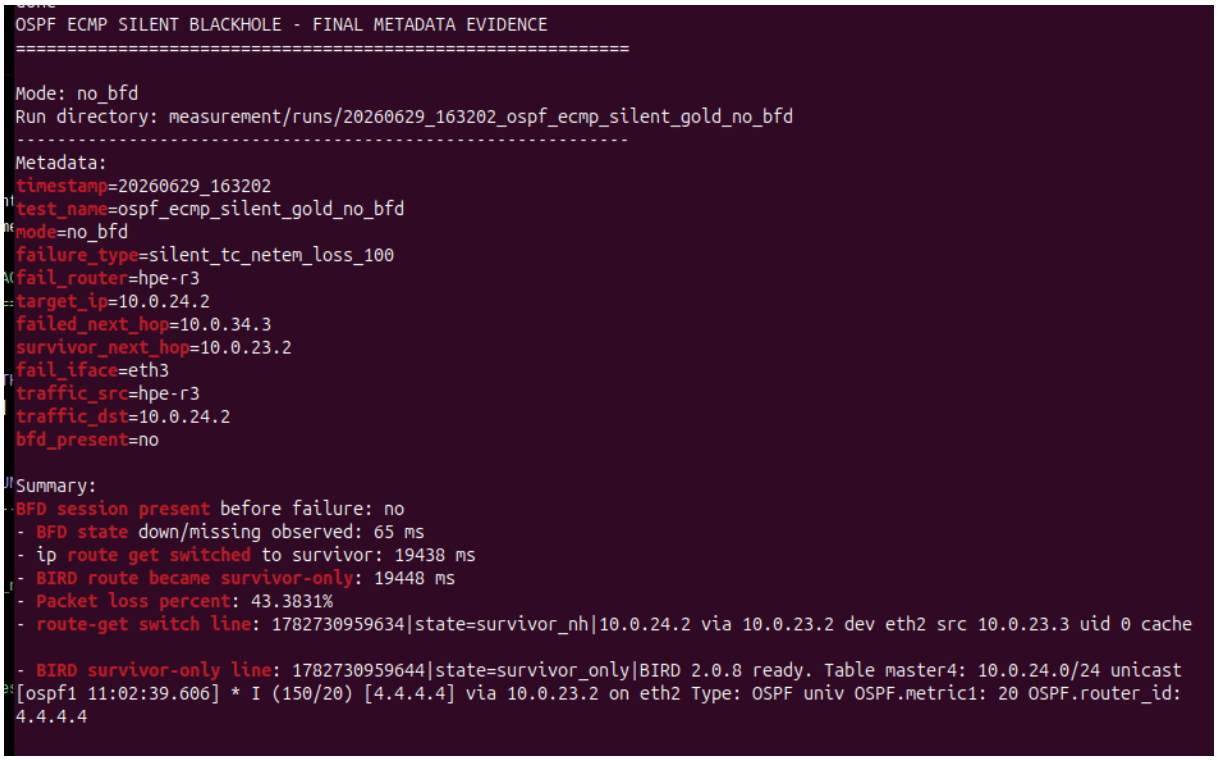


Figure 7.1: OSPF ECMP silent blackhole gold metadata showing failed next-hop 10.0.34.3 and surviving next-hop 10.0.23.2

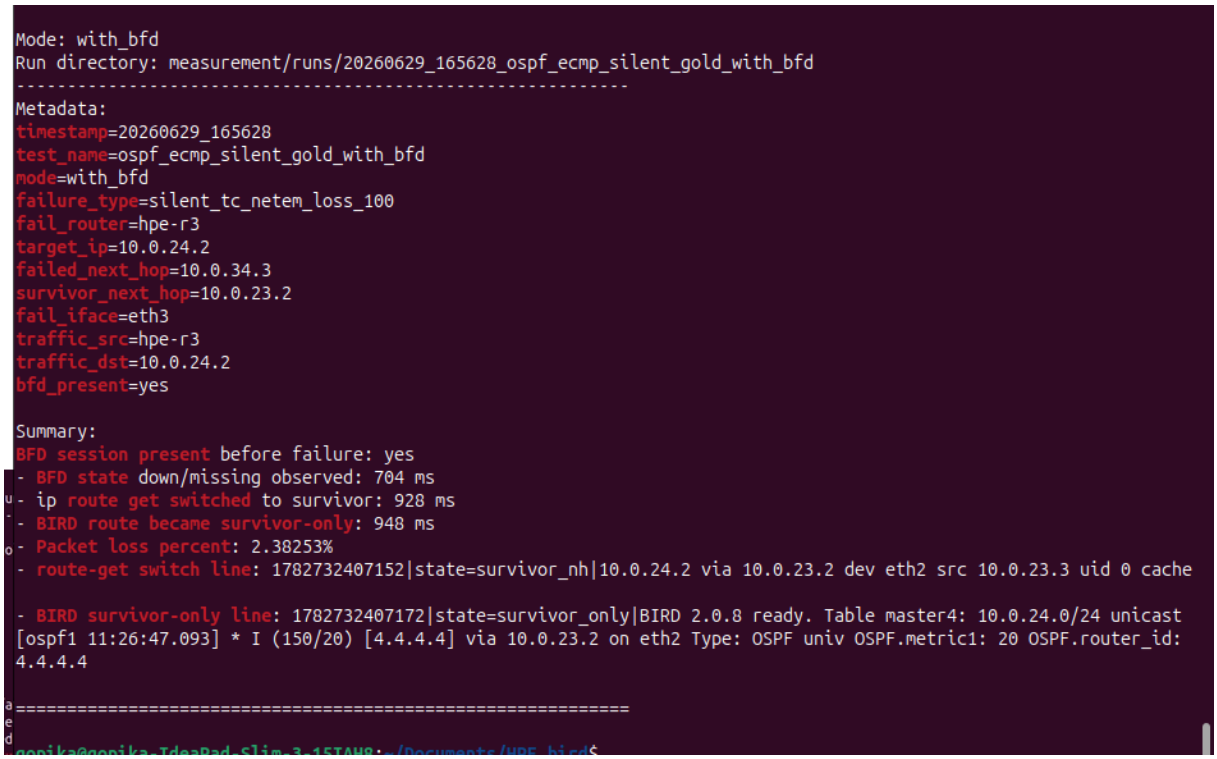


Figure 7.2: OSPF ECMP silent blackhole – forwarding state before failure injection

7.2 Why Silent Failure Matters

Without fast failure detection, OSPF continues using the failed next-hop until protocol timers expire. With BFD enabled, BFD packets detect loss of connectivity even when the interface still appears up, triggering fast recovery.

7.3 Results (15 runs per mode, 30 total)

Table 7.1: OSPF ECMP silent blackhole comparison: no-BFD vs with-BFD

Metric	No-BFD Avg	With-BFD Avg	Improvement
Kernel route event (ms)	18001.07	1253.60	93.04% faster
Linux forwarding switch (ms)	17989.40	1264.60	92.97% faster
BIRD route update (ms)	17994.20	1266.33	92.96% faster
Traffic outage (ms)	12369.80	1126.00	90.90% lower
Traffic loss (%)	41.81	3.48	91.68% lower

```
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird

python3 scripts/54_summarize_ospf_ecmp_silent_gold.py
================================================================================================================
OSPF ECMP SILENT FAILURE GOLD SUMMARY
================================================================================================================

Mode: no_bfd | Runs: 15
----------------------------------------------------------------------------------------------------------------
Metric                              Count      Average       Median          Min          Max       StdDev
bfd_detect_ms                           -      ignored      ignored            -            -            -
kernel_event_ms                        15     18001.07     17951.00     16138.00     20100.00      1220.63
route_get_switch_ms                    15     17989.40     17941.00     16093.00     20132.00      1225.63
bird_route_survivor_only_ms            15     17994.20     17943.00     16097.00     20136.00      1225.51
traffic_outage_ms                      15     12369.80     12095.00      6978.00     18488.00      3747.58
traffic_loss_percent                   15        41.81        43.08        25.43        57.19         9.90
bfd_packet_lines                       15         0.00         0.00         0.00         0.00         0.00

Mode: with_bfd | Runs: 15
----------------------------------------------------------------------------------------------------------------
Metric                              Count      Average       Median          Min          Max       StdDev
bfd_detect_ms                          15       706.07       695.00       641.00       834.00        46.63
kernel_event_ms                        15      1253.60      1231.00       873.00      1643.00       276.38
route_get_switch_ms                    15      1264.60      1194.00       921.00      1702.00       279.23
bird_route_survivor_only_ms            15      1266.33      1196.00       868.00      1710.00       286.04
traffic_outage_ms                      15      1126.00      1094.00       745.00      1535.00       274.74
traffic_loss_percent                   15         3.48         3.37         2.31         4.75         0.85
bfd_packet_lines                       15      2576.00      2573.00      2522.00      2628.00        33.58

Comparison: no-BFD vs with-BFD
----------------------------------------------------------------------------------------------------------------
kernel_event_ms                 no_bfd_avg=  18001.07, with_bfd_avg=   1253.60, improvement=93.04% lower/faster
 with BFD
route_get_switch_ms             no_bfd_avg=  17989.40, with_bfd_avg=   1264.60, improvement=92.97% lower/faster
 with BFD
bird_route_survivor_only_ms     no_bfd_avg=  17994.20, with_bfd_avg=   1266.33, improvement=92.96% lower/faster
 with BFD
traffic_outage_ms               no_bfd_avg=  12369.80, with_bfd_avg=   1126.00, improvement=90.90% lower/faster
 with BFD
traffic_loss_percent            no_bfd_avg=     41.81, with_bfd_avg=      3.48, improvement=91.68% lower/faster
 with BFD

Professional wording:
----------------------------------------------------------------------------------------------------------------
In the silent ECMP branch failure experiment, the interface was kept UP while packets on one ECMP branch were silen
tly dropped using tc/netem. Without BFD, the Linux forwarding decision switched to the surviving path in 17989.40 m
s on average, and BIRD route-table sampling observed the survivor-only route in 17994.20 ms on average. With OSPF-B
FD enabled, BFD failure was observed in 706.07 ms on average, the forwarding decision switched in 1264.60 ms on ave
rage, and BIRD observed the survivor-only route in 1266.33 ms on average. Traffic outage reduced from 12369.80 ms t
o 1126.00 ms, and packet loss reduced from 41.81% to 3.48%. This shows that BFD is especially valuable for silent b
lackhole-style failures where the kernel does not receive an immediate link-down event.
================================================================================================================
```

Figure 7.3: OSPF ECMP silent blackhole gold summary comparing no-BFD and with-BFD recovery

7.4 Conclusion

BFD reduced the average forwarding switch time from **17989 ms to 1264 ms**. Traffic outage reduced from **12369 ms to 1126 ms**. Packet loss reduced from **41.81% to 3.48%**. This confirms that OSPF-BFD integration is essential for detecting silent blackhole failures where the interface does not go down.

Chapter 8

Multihop BFD Failure Detection

8.1 Objective

The objective was to validate **multihop BFD failure detection** across a routed path, where BFD peers are separated by one or more intermediate routers. Target: detection time below 1 second.

8.2 Multihop BFD Session

Table 8.1: Multihop BFD session endpoints

Endpoint	Router	Peer Address
Endpoint 1	`hpe-r1`	10.0.82.3
Endpoint 2	`hpe-r8`	10.0.14.2

BFD interval: 0.100 s BFD timeout: 0.500 s Interface field: `---` (confirms non-direct multihop session).

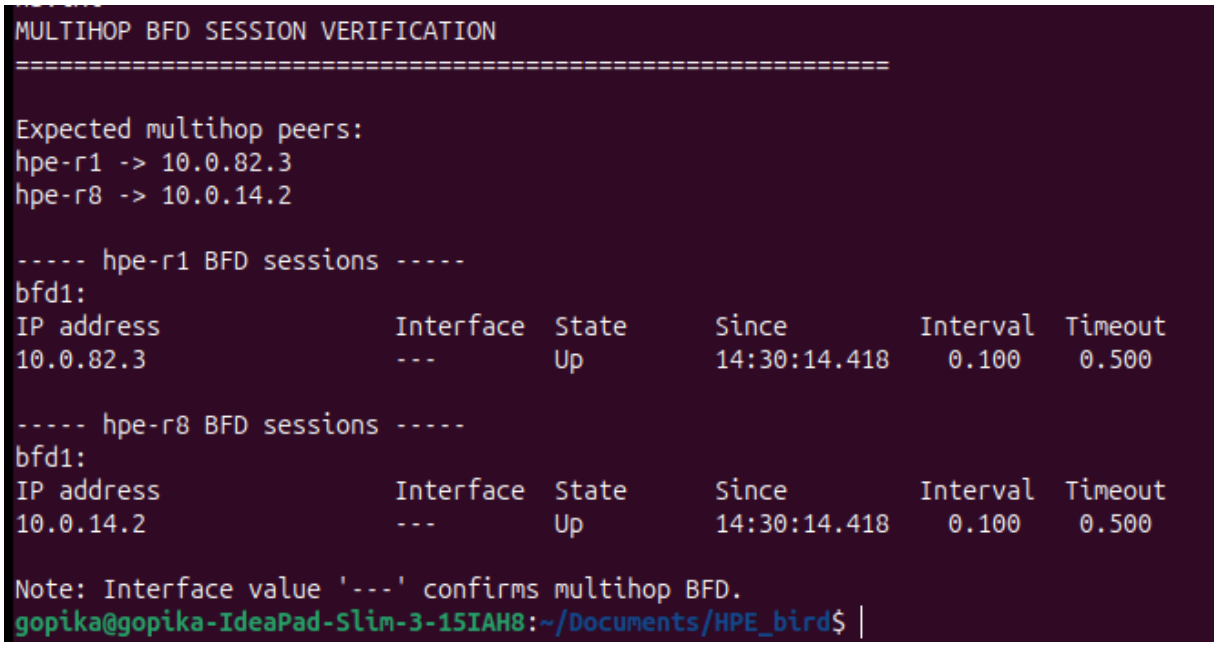


Figure 8.1: Multihop BFD session established between `hpe-r1` and `hpe-r8`

8.3 Routed Path and Failure Point

The active routed path for the multihop BFD session was:

```
hpe-r1 -> hpe-r4 -> hpe-r7 -> hpe-r8
```

The corrected failure point was `hpe-r4 eth3` → `hpe-r7 eth1`.

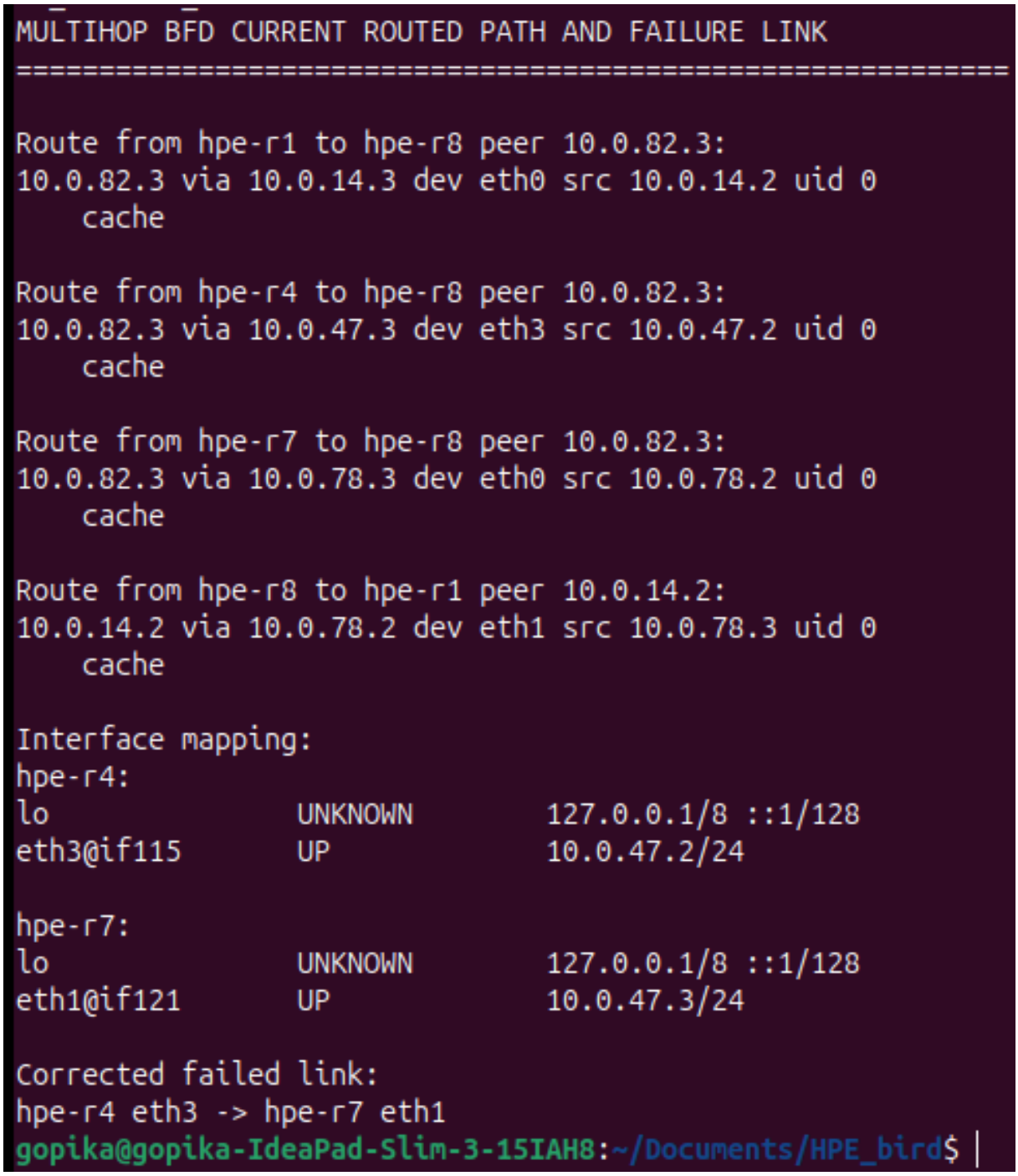


Figure 8.2: Current routed path used by the multihop BFD session, crossing the hpe-r4 eth3 to hpe-r7 eth1 link

8.4 Results (15 runs)

Table 8.2: Multihop BFD detection summary

Metric	hpe-r1	hpe-r8
Average detection time	511.27 ms	570.73 ms
Maximum detection time	581 ms	642 ms
Target < 1 s passed	15/15	15/15
Timeouts	0/15	0/15

```
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cat screenshots/multihop_bfd_final/03_multihop_bfd_pilot_
result.txt
REPRESENTATIVE MULTIHOP BFD PILOT RUN
============================================================

Timestamp: 20260629_194605
hpe-r1 peer: 10.0.82.3
hpe-r8 peer: 10.0.14.2
We fail the routed path by bringing down hpe-r4 eth3.
Ping log: evidence/multihop_bfd/multihop_bfd_ping_20260629_194605.log
Failure time ms: 1782742566749
hpe-r1 multi-hop BFD detection time: 496 ms
hpe-r8 multi-hop BFD detection time: 564 ms
10.0.82.3                ---        Down       14:16:07.287    1.000    0.000
10.0.14.2                ---        Down       14:16:07.256    1.000    0.000
Ping loss: 69.1304%
11. CSV result
20260629_194605,multihop_bfd_failure,hpe-r4-eth3_to_hpe-r7-eth1,10.0.82.3,10.0.14.2,496,564,5874,2,230,71,69.1304%,
yes
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$
```

Figure 8.3: Multihop BFD failure detection result – intermediate router interface down observed

```
MULTIHOP BFD FAILURE DETECTION SUMMARY
============================================================================================
Runs: 15
Path: hpe-r1 -> hpe-r4 -> hpe-r7 -> hpe-r8
Failed link: hpe-r4 eth3 -> hpe-r7 eth1
Target: detection < 1 second

Metric                                Average       Median        Min        Max      StdDev
--------------------------------------------------------------------------------------------
hpe-r1 BFD detection                  511.27 ms     501.00 ms   488.00 ms   581.00 ms    22.43 ms
hpe-r8 BFD detection                  570.73 ms     567.00 ms   544.00 ms   642.00 ms    24.03 ms

Target < 1 second passed: 15/15 runs
Timeout runs: 0/15

Conclusion:
Multihop BFD detected the routed-path failure within 1 second in 15/15 runs. Average detection was 511.27 ms from h
pe-r1 and 570.73 ms from hpe-r8.
============================================================================================
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$
```

Figure 8.4: Final multihop BFD failure detection summary for 15 corrected runs – sub-second detection from both endpoints

8.5 Packet Capture Evidence

Figure 8.5: Packet capture showing BFDv1 multihop packets between 10.0.82.3 and 10.0.14.2 on UDP port 4784

8.6 Conclusion

Across 15 corrected runs, multihop BFD detected the failure in under 1 second in every run (target passed 15/15). Average detection was 511 ms from `hpe-r1` and 570 ms from `hpe-r8`. This validates BFD for detecting failures across routed multihop paths.

Chapter 9

OSPF Area Healing Across ABRs (Silent Blackhole)

9.1 Objective

The objective was to validate **OSPF inter-area healing across ABRs** under a silent failure condition, and to compare recovery with and without BFD.

Traffic tested: `hpe-h1` → `hpe-h2` (10.0.61.2 → 10.0.82.2).

The failed logical path was:

```
hpe-r3 eth3 (10.0.34.2/24) -> hpe-r4 eth2 (10.0.34.3/24)
```

9.2 Baseline and BFD Precheck

Before failure, the baseline forwarding decision on `hpe-r3` was:

```
10.0.82.2 via 10.0.34.3 dev eth3
```

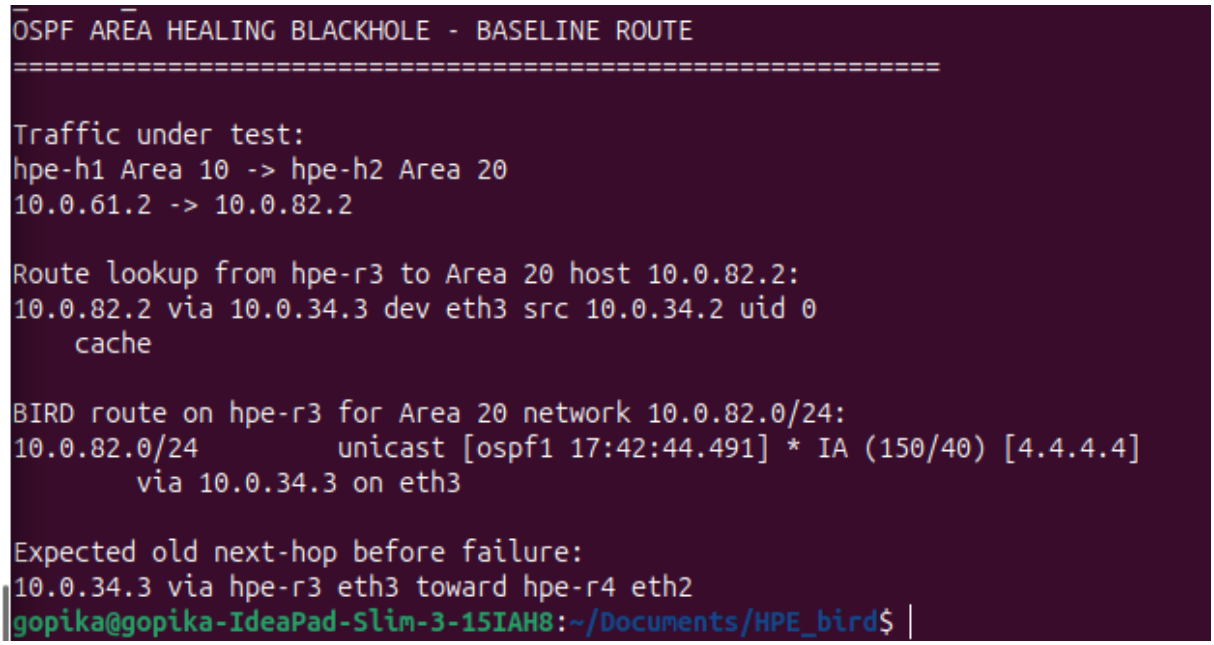


Figure 9.1: Baseline route from hpe-r3 to Area 20 host network before silent failure injection

BFD was verified active on the OSPF ABR link before the with-BFD test:

```
hpe-r3: 10.0.34.3  eth3  Up  interval 0.100  timeout 0.500
hpe-r4: 10.0.34.2  eth2  Up  interval 0.100  timeout 0.500
```

```
OSPF AREA HEALING BLACKHOLE - BFD PRECHECK
============================================================

Tested OSPF ABR/core link:
hpe-r3 eth3 = 10.0.34.2/24
hpe-r4 eth2 = 10.0.34.3/24

BFD session on hpe-r3 toward hpe-r4:
bfd1:
IP address                Interface  State      Since         Interval  Timeout
10.0.34.3                 eth3       Up         17:43:14.732    0.100    0.500

BFD session on hpe-r4 toward hpe-r3:
bfd1:
IP address                Interface  State      Since         Interval  Timeout
10.0.34.2                 eth2       Up         17:43:14.732    0.100    0.500

OSPF neighbour on hpe-r3 toward hpe-r4:
ospf1:
Router ID       Pri          State      DTime   Interface  Router IP
4.4.4.4           1     Full/DR         18.436  eth3       10.0.34.3

OSPF neighbour on hpe-r4 toward hpe-r3:
ospf1:
Router ID       Pri          State      DTime   Interface  Router IP
3.3.3.3           1     Full/BDR        16.803  eth2       10.0.34.2
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$
```

Figure 9.2: BFD precheck – active BFD sessions on the hpe-r3 to hpe-r4 OSPF link before silent failure injection

9.3 Results

Table 9.1: OSPF area-healing silent blackhole: with-BFD vs no-BFD

Metric	With BFD	Without BFD	Improvement
BFD detection	573 ms	N/A	—
Route-get switch	1313 ms	18761 ms	≈93%
Kernel route update	1315 ms	18762 ms	≈93%
BIRD route update	1250 ms	18835 ms	≈93%
Packet loss	2%	30%	≈93%

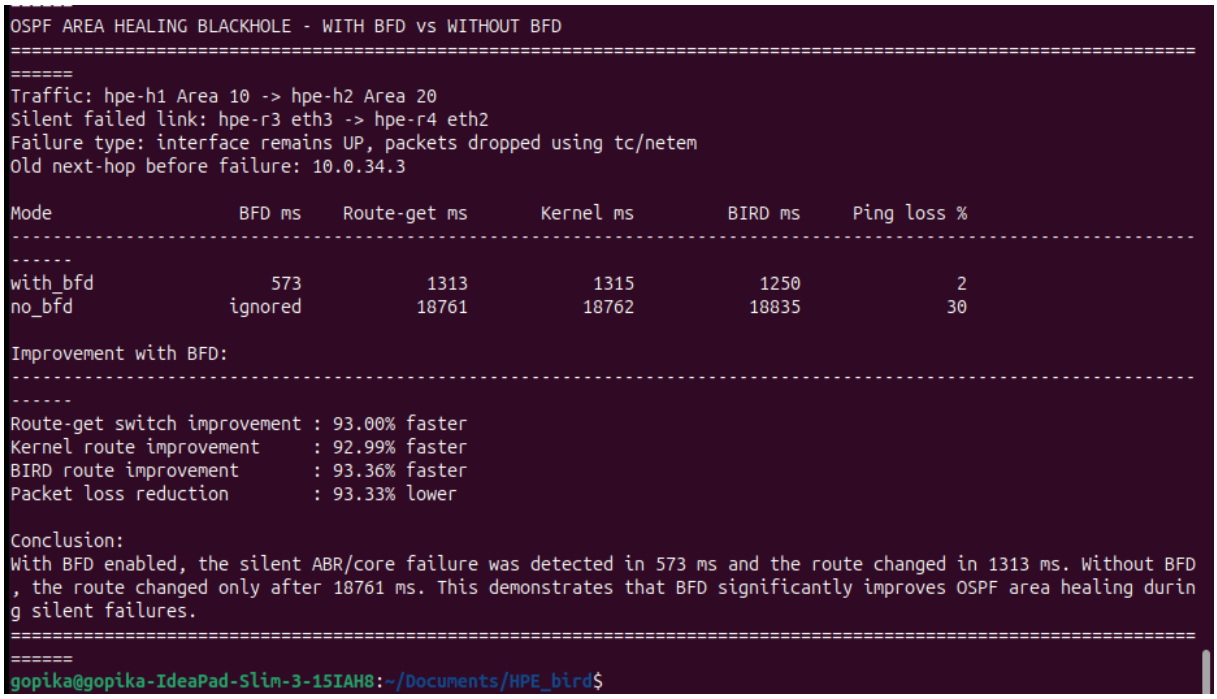


Figure 9.3: OSPF area-healing comparison under silent blackhole failure – with-BFD vs no-BFD

9.4 Conclusion

Without BFD, OSPF took approximately 18.7 seconds to recover from a silent blackhole failure on the ABR link, with 30% packet loss. With BFD enabled, failure was detected in 573 ms, the route changed in ∼1.3 seconds, and packet loss reduced to 2%. This confirms that BFD significantly improves OSPF area healing during silent failures where the interface remains up.

Chapter 10

OSPF NSSA Type-7 to Type-5 Translation and Stub Area Containment

10.1 Objective

The objective was to validate that an external route originated inside the NSSA area is advertised as a Type-7 LSA inside the NSSA and translated into a Type-5 LSA by the ABR for the backbone area, and that the external route is not directly flooded into the stub area.

10.2 Test Route

A controlled static blackhole route was created on `hpe-r6`:

```
172.16.66.0/24 blackhole
```

This route was exported into OSPF from inside NSSA Area 10.

10.3 Type-7 LSA Evidence Inside NSSA

```
hpe-r6: 0007 172.16.66.255 6.6.6.6
hpe-r5: 0007 172.16.66.255 6.6.6.6
hpe-r3: 0007 172.16.66.255 6.6.6.6
```

```
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird

for r in hpe-r6 hpe-r5 hpe-r3; do
  echo
  echo "========== $r LSADB type 7 =========="
  docker exec "$r" birdc show ospf lsadb type 7 | grep -E "Type|172.16.66.255|0007"
done

========== hpe-r6 LSADB type 7 ==========
 Type    LS ID           Router          Sequence   Age  Checksum
 0007  172.16.66.255    6.6.6.6         80000002   277    a283

========== hpe-r5 LSADB type 7 ==========
 Type    LS ID           Router          Sequence   Age  Checksum
 0007  172.16.66.255    6.6.6.6         80000002   276    a283

========== hpe-r3 LSADB type 7 ==========
 Type    LS ID           Router          Sequence   Age  Checksum
 0007  172.16.66.255    6.6.6.6         80000002   278    a283
```

Figure 10.1: Type-7 LSA evidence inside NSSA Area 10, originated by router ID 6.6.6.6

10.4 Type-5 Translation Evidence Outside NSSA

At the ABR `hpe-r3` (router ID 3.3.3.3), the Type-7 LSA was translated into a Type-5 LSA for the backbone area:

```
hpe-r3: 0005 172.16.66.255 3.3.3.3
hpe-r1: 0005 172.16.66.255 3.3.3.3
hpe-r2: 0005 172.16.66.255 3.3.3.3
hpe-r4: 0005 172.16.66.255 3.3.3.3
```

```
0007  172.16.66.255   6.6.6.6          80000002   278    a283
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird

for r in hpe-r3 hpe-r1 hpe-r2 hpe-r4; do
  echo
  echo "========== $r LSADB type 5 =========="
  docker exec "$r" birdc show ospf lsadb type 5 | grep -E "Type|172.16.66.255|0005"
done

========== hpe-r3 LSADB type 5 ==========
Type   LS ID           Router          Sequence   Age  Checksum
0005  0.0.0.0         1.1.1.1         80000004   604    239e
0005  0.0.0.0         2.2.2.2         80000006   164    01ba
0005  172.16.66.255   3.3.3.3         80000001   278    75c5

========== hpe-r1 LSADB type 5 ==========
Type   LS ID           Router          Sequence   Age  Checksum
0005  0.0.0.0         1.1.1.1         80000004   604    239e
0005  0.0.0.0         2.2.2.2         80000006   164    01ba
0005  172.16.66.255   3.3.3.3         80000001   280    75c5

========== hpe-r2 LSADB type 5 ==========
Type   LS ID           Router          Sequence   Age  Checksum
0005  0.0.0.0         1.1.1.1         80000004   604    239e
0005  0.0.0.0         2.2.2.2         80000006   163    01ba
0005  172.16.66.255   3.3.3.3         80000001   280    75c5

========== hpe-r4 LSADB type 5 ==========
Type   LS ID           Router          Sequence   Age  Checksum
0005  0.0.0.0         1.1.1.1         80000004   604    239e
0005  0.0.0.0         2.2.2.2         80000006   164    01ba
0005  172.16.66.255   3.3.3.3         80000001   280    75c5
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$
```

Figure 10.2: Type-5 LSA evidence outside the NSSA, originated by ABR router ID 3.3.3.3

10.5 External Route Containment in Stub Area

On stub-area router `hpe-r8`, BIRD reported `Network not found` for `172.16.66.0/24`. However, the Linux kernel still had forwarding capability through the default route:

```
172.16.66.1 via 10.0.78.2 dev eth0
```

```
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird

echo "hpe-r8 does not have the external route in BIRD:"
docker exec hpe-r8 birdc show route 172.16.66.0/24 all

echo
echo "hpe-r8 still has a default path for forwarding:"
docker exec hpe-r8 ip route get 172.16.66.1
hpe-r8 does not have the external route in BIRD:
BIRD 2.0.8 ready.
Network not found

hpe-r8 still has a default path for forwarding:
172.16.66.1 via 10.0.78.2 dev eth0 src 10.0.78.3 uid 0
    cache
```

Figure 10.3: Stub area containment on hpe-r8 – external route absent in BIRD, forwarding possible via default route

10.6 Conclusion

NSSA Type-7 to Type-5 translation and stub-area containment worked correctly. The external route was contained inside Area 10 as Type-7, translated at ABR `hpe-r3` into Type-5, made visible to backbone routers, and correctly blocked from direct flooding into the stub area.

Chapter 11

BGP Graceful Restart, LLGR and Peer Flap Validation

11.1 Objective

This experiment validated BGP control-plane resilience: Graceful Restart forwarding retention, LLGR stale-route timer behaviour, and BGP peer flap recovery.

BGP prefix used: `10.0.93.0/24` Host: `10.0.93.2`

```
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird

docker exec hpe-r1 birdc show route 10.0.93.0/24 all
docker exec hpe-r1 ip route get 10.0.93.2
BIRD 2.0.8 ready.
Table master4:
10.0.93.0/24         unicast [r9 2026-06-29] * (100) [AS65002i]
        via 10.0.19.3 on eth1
        Type: BGP univ
        BGP.origin: IGP
        BGP.as_path: 65002
        BGP.next_hop: 10.0.19.3
        BGP.local_pref: 100
                     unicast [r2 2026-06-29] (100) [AS65002i]
        via 10.0.12.3 on eth2
        Type: BGP univ
        BGP.origin: IGP
        BGP.as_path: 65002
        BGP.next_hop: 10.0.12.3
        BGP.local_pref: 100
10.0.93.2 via 10.0.19.3 dev eth1 src 10.0.19.2 uid 0
    cache
```

Figure 11.1: Baseline BGP reachability to the test prefix 10.0.93.0/24

11.2 Graceful Restart Forwarding Retention

To prove GR rather than ordinary BGP failover, a blackhole guard was installed and the alternate `hpe-r1` → `hpe-r2` path was disabled, so that continued forwarding must come

from retained BGP state alone.

Table 11.1: BGP Graceful Restart forwarding result

Measurement	Result
BGP non-established observed	114 ms
Old next-hop retained after restart	Yes
Alternate next-hop used	No
Blackhole/unreachable observed	No
Ping transmitted	200
Ping received	200
Packet loss	0%

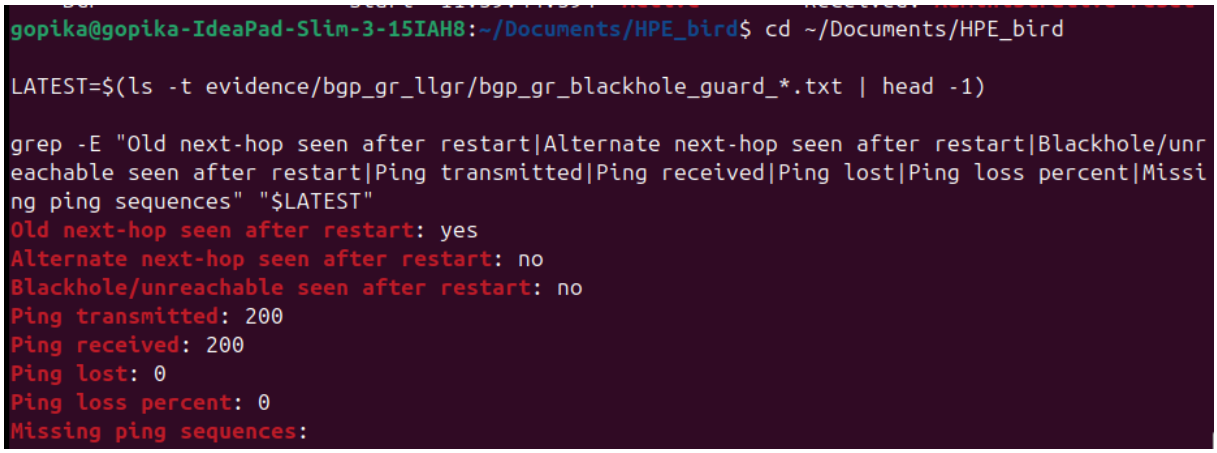


Figure 11.2: Forwarding continued through the retained old next-hop – no fallback to alternate path

11.3 LLGR Stale Route Timer Behaviour

Timers used: GR timer = 5 s, LLGR stale timer = 10 s.

Table 11.2: LLGR stale route timer result

Measurement	Result
First stale marker	5183 ms (matches GR timer)
First route missing time	15239 ms (matches GR + LLGR window)
Ping transmitted	273
Ping received	273
Packet loss	0%

```
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$ cd ~/Documents/HPE_bird

column -s, -t results/gr_llgr_triangle/gr_llgr_triangle.csv
timestamp        router  target_prefix  gr_time_s  llgr_stale_time_s  first_stale_marker_ms  first_route_missing_ms  first_kernel_route_missing_ms  ping_tx  ping_rx  ping_loss_percent
20260626_104011  hpe-r1  10.0.93.0/24   5          10                 5183                   15239                   NA                             273      273      0
20260626_104011  hpe-r2  10.0.93.0/24   5          10                 5183                   15239                   NA                             273      273      0
gopika@gopika-IdeaPad-Slim-3-15IAH8:~/Documents/HPE_bird$
```

Figure 11.3: LLGR stale-route timer result – stale marker at ∼5 s, route withdrawal at ∼15 s

11.4 BGP Peer Flap Behaviour

The BGP peer between `hpe-r1` and `hpe-r9` was intentionally disabled and re-enabled to test flap recovery.

Table 11.3: BGP peer flap result

Measurement	**Result**
Flapped router	`hpe-r1`
Flapped peer	r9
Route moved to alternate path	93 ms
BGP re-established	1596 ms
Route returned to direct path	1754 ms
Ping transmitted	162
Ping received	162
Packet loss	0%

11.5 Conclusion

Graceful Restart preserved forwarding during temporary BGP control-plane disruption with **0% packet loss**. LLGR retained the stale route for the configured timer window (stale at ∼5 s, withdrawn at ∼15 s). The peer flap test showed the network moved traffic to an alternate path in 93 ms and returned to the direct path in 1754 ms with 0% loss. Overall, the BGP edge design provided control-plane resilience and stable forwarding during restart and flap scenarios.

Chapter 12

Graphical Summary and Comparative Analysis

12.1 Objective

This section summarizes major timing and recovery results. The graphs compare BFD detection times, OSPF convergence behaviour, packet loss, and BGP resilience timers to show the overall impact of BFD-based failure detection.

12.2 BFD Detection Time Comparison

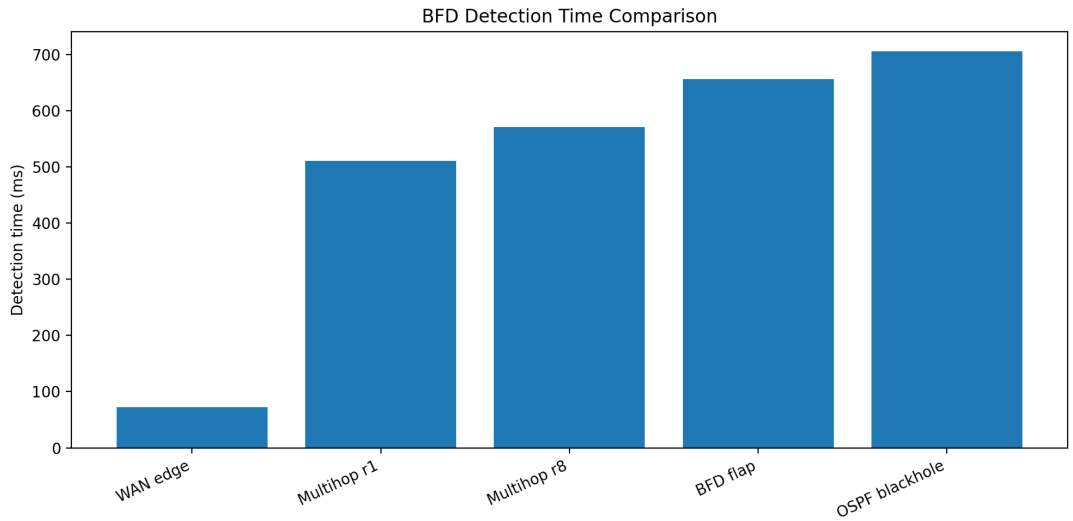


Figure 12.1: BFD detection time comparison across WAN-edge, multihop, repeated flap, and OSPF silent blackhole scenarios

The WAN-edge BFD test showed the fastest detection ($\sim$72 ms average). Multihop and silent-failure cases had higher detection times due to indirect paths or blackhole-style failures. The repeated flap test confirmed stability across all 5 cycles.

12.3 OSPF Silent Blackhole Recovery: Without BFD vs With BFD

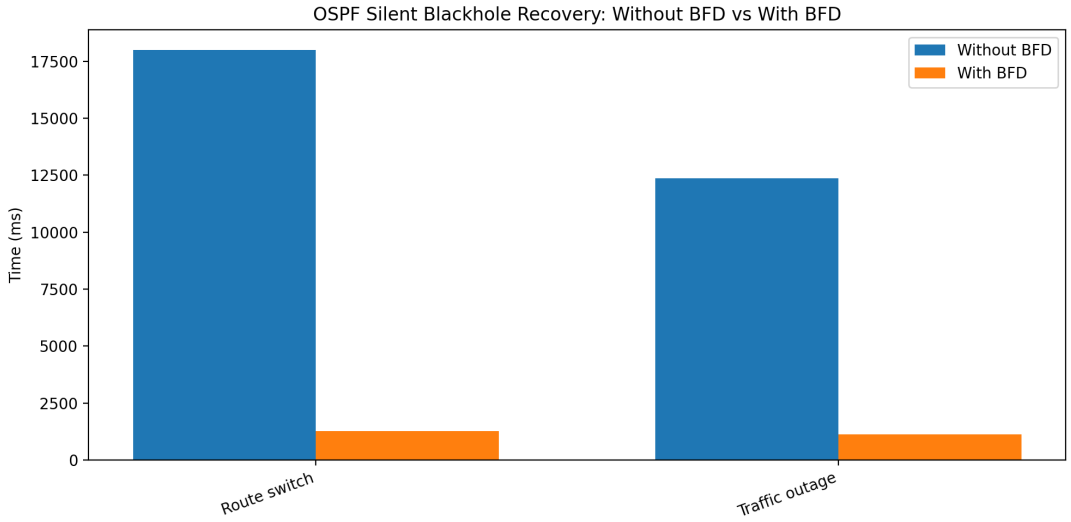


Figure 12.2: OSPF silent blackhole recovery time comparison – without BFD vs with BFD

Table 12.1: Silent blackhole recovery time

Metric	Without BFD	With BFD
Route switch time	17989.40 ms	1264.60 ms
Traffic outage	12369.80 ms	1126.00 ms

12.4 Packet Loss Improvement in Silent Failure

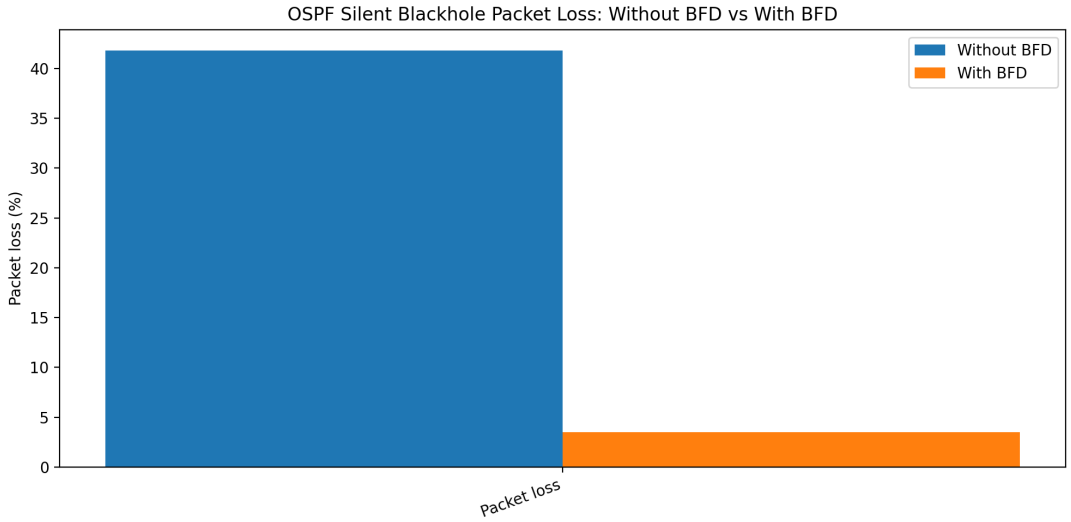


Figure 12.3: Packet loss comparison for OSPF silent blackhole recovery

Without BFD: **41.81%** packet loss. With BFD: **3.48%** packet loss. Faster failure detection directly improved data-plane recovery.

12.5 OSPF Area Healing Comparison

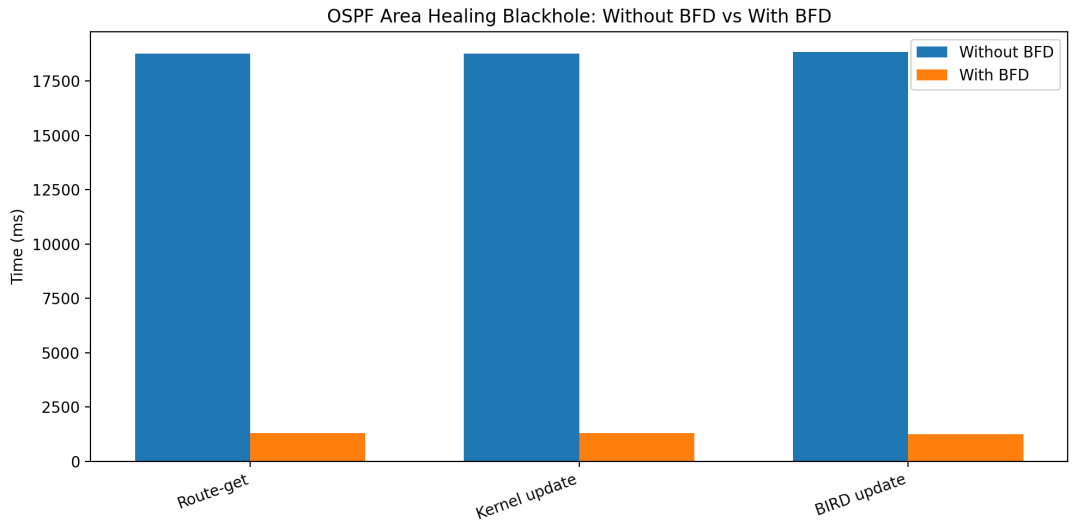


Figure 12.4: OSPF area-healing recovery comparison – without BFD vs with BFD

BFD reduced the route update delay from ∼18 seconds to ∼1.3 seconds and packet loss from 30% to 2% during inter-area ABR healing.

12.6 Overall Comparative Analysis

BFD tuning had the greatest impact in silent failure scenarios. In direct interface-down failures, the Linux kernel detects failure quickly through link-state events. In blackhole-style failures, the interface remains up, so normal protocol convergence takes much longer. BFD detected these silent failures faster, reducing route switch time, traffic outage, and packet loss.

BGP Graceful Restart retained routes during daemon restart, while LLGR retained stale routes for the configured timer window before removal. Together, these results confirm that the project successfully demonstrated dynamic failure detection, faster convergence, route retention, and recovery.

Chapter 13

Part I Conclusion

This project successfully implemented and validated a self-healing dynamic routing architecture using OSPF, BGP, BFD, ECMP, Graceful Restart, and LLGR in the BIRD routing stack.

The network was able to detect failures, reroute traffic, preserve routing state, and recover from both link failures and control-plane disruptions. BFD provided the greatest improvement in silent blackhole failures, where the interface remained up but packets were dropped – reducing convergence delay, traffic outage, and packet loss significantly.

Key results summary:

- BFD WAN edge detection: **72 ms average**, well below the 300 ms target.
- OSPF core link recovery: **85 ms** kernel forwarding switch, **635 ms** full convergence.
- OSPF ECMP direct failover: **∼75 ms** with 0% packet loss.
- OSPF ECMP silent blackhole with BFD: forwarding switch improved from **17989 ms to 1264 ms**.
- Multihop BFD: sub-second detection, **15/15** runs passed.
- OSPF area healing with BFD: route recovery in **∼1.3 s** vs **18.7 s** without BFD.
- BGP Graceful Restart: **0% packet loss** during control-plane restart.
- BGP peer flap: alternate path in **93 ms**, full recovery in **1754 ms**.

The implementation provides measurable evidence for detection time, convergence behaviour, packet loss reduction, and route-retention capability. The project demonstrates that combining fast failure detection with resilient routing mechanisms produces a network that is fault-tolerant, adaptive, and reliable.

Part II

BGPoST Experiment Report: Secure Transport for BGP

Chapter 14

Introduction

Border Gateway Protocol (BGP) is used to exchange routing information between routers. Traditionally, BGP runs over plain TCP without encryption. Secure transport mechanisms such as TLS, QUIC, and TCP-AO-style authentication improve BGP security.

This experiment studies BGP route propagation behaviour across five transport modes using a Docker-based BGPoST lab: **TCP, TLS, QUIC, TLS-AO Static,** and **TLS-AO Dynamic**.

Chapter 15

Objective

The main objective is to compare the **prefix propagation delay** of BGP when different secure transport mechanisms are used – checking how quickly BGP prefixes propagate through a chain of routers when the underlying transport mode is changed.

Chapter 16

Experimental Setup

Docker containers were used, each running the BIRD routing daemon. ExaBGP was the route injector. Topology: ten BGP routers in a chain/loop: ExaBGP $\rightarrow R_1 \rightarrow R_2 \rightarrow \cdots \rightarrow R_{10} \rightarrow R_1$.

Table 16.1: Prefix propagation experiment parameters

Parameter	**Value**
Number of routers	10
Number of observed prefixes	13,104
Announcement interval	50 ms
Link delay	15 ms
Routing daemon	BIRD
Route injector	ExaBGP
Output format	MRT logs

Chapter 17

Methodology

For each transport mode: (1) Docker networks created, (2) containers started with BIRD configs, (3) BGP sessions waited to reach **Established**, (4) ExaBGP injected prefixes into R_1, (5) BGP updates recorded as MRT files, (6) MRT files parsed to calculate propagation delay.

Chapter 18

Prefix Propagation Results and Discussion

All five transport modes established BGP sessions and propagated prefixes through the 10-router topology.

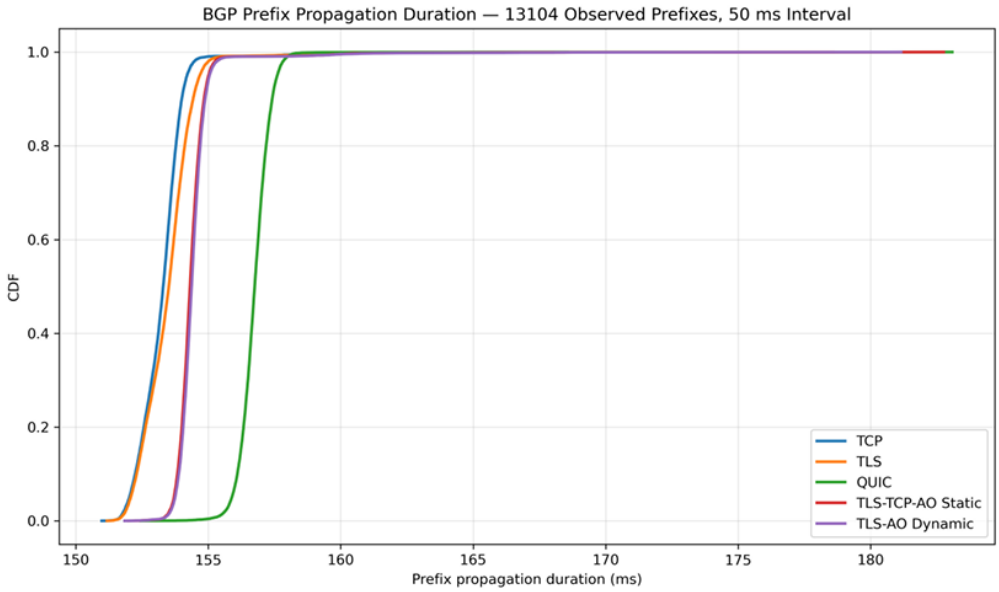


Figure 18.1: CDF comparison of prefix propagation delay for 13,104 observed prefixes across TCP, TLS, QUIC, TLS-AO Static, and TLS-AO Dynamic transport modes

- **TCP and TLS** showed very similar propagation – adding TLS did not introduce large delay.
- **TLS-AO Static and TLS-AO Dynamic** also stayed close to TCP and TLS.
- **QUIC** showed slightly higher propagation delay, but still acceptable compared to total path delay.

The main reason for overall propagation delay is the configured 15 ms link delay per hop, which accumulates across the 10-router chain.

Conclusion: Secure transport mechanisms can be used with BGP while maintaining acceptable route propagation performance. Secure BGP transport improves communication security without causing a major increase in prefix propagation delay.

Chapter 19

Multi-Homing with BGP over TLS

19.1 Network Topology

The topology consists of three Autonomous Systems deployed as Docker containers: AS1 (Provider, backup path, AS 65001), AS2 (Provider, main path, AS 65002), and AS3 (Stub, multihomed customer, AS 65003). All BGP sessions between the three routers are established over TLS using mutual X.509 certificate authentication, implemented using the BGPoTLS fork of the BIRD routing daemon. Each router presents a certificate signed by a common lab Certificate Authority, ensuring that only authorized routers can establish BGP sessions.

Table 19.1: AS roles in multi-homing experiment

Router	**Role**
AS1 (65001)	Provider – backup path, always connected to both AS2 and AS3
AS2 (65002)	Provider – main path, AS3's primary internet connection
AS3 (65003)	Stub, multihomed customer network

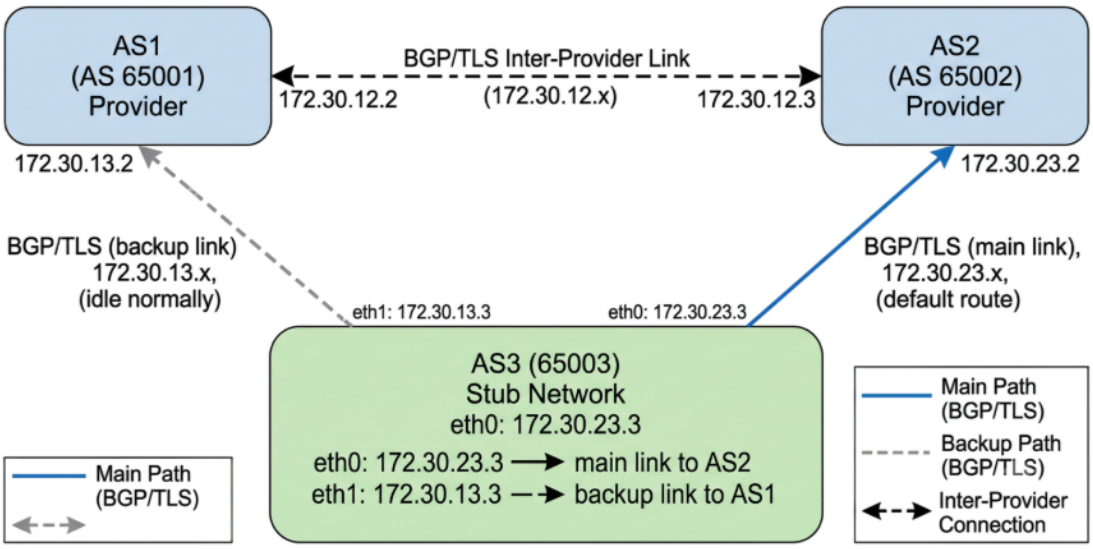


Figure 19.1: Multi-homing topology showing AS1 (backup provider), AS2 (main provider), and AS3 (multihomed customer) with primary and backup BGP over TLS paths

19.2 Normal Operation

Under normal operation, AS3 connects to the internet through AS2 via its primary interface (`eth0`, subnet `172.30.23.0/24`). The BGP session between AS3 and AS2 is active and in the **Established** state, and AS3's default route points directly to AS2 at `172.30.23.2`. AS3 also maintains a secondary BGP session with AS1 over a backup interface (`eth1`, subnet `172.30.13.0/24`), but this link carries no data plane traffic under normal conditions.

19.3 BGP over TLS with Mutual Authentication

Every BGP session uses TLS 1.3 with mutual X.509 certificates:

```
transport tls;
tls certificate "/certs/router.crt";
tls pkey "/certs/router.key";
tls root ca "/certs/ca.crt";
```

19.4 Certificate-Embedded Configuration

The novel aspect of this implementation is that AS3's X.509 certificate carries a custom configuration block embedded in a private OID extension field. This JSON block specifies the backup tunnel parameters – the GRE tunnel endpoints, the address of AS1 used as the transit path, the prefix to protect, and the probe interval.

```
{
  "prefixes": ["10.3.0.0/16"],
  "tunnel": {
```

```
    "type": "GRE",
    "local_addr": "172.30.23.3",
    "remote_addr": "172.30.23.2",
    "backup_via": "172.30.13.2",
    "keepalive_interval": 5
  },
  "as_number": 65003
}
```

Because this JSON is inside a cryptographically signed certificate, it cannot be tampered with without invalidating the certificate.

19.5 Automatic Failover via Tunnel Manager

A Python script called `tunnel_manager.py` runs alongside BIRD inside AS3's container. On startup it reads this certificate configuration and begins monitoring the main link to AS2 by sending ICMP probe packets to `172.30.23.2` every five seconds via `eth0`.

When the main link between AS3 and AS2 fails, `tunnel_manager.py` detects three consecutive failed probes within fifteen seconds and automatically activates the backup path. It performs the following steps without any manual operator intervention:

1. Adds a static host route directing traffic for `172.30.23.2` via AS1 through `eth1`.
2. Creates a GRE tunnel interface named `gre-backup` with local endpoint `172.30.23.3` and remote endpoint `172.30.23.2`.
3. Installs routes for AS3's prefix through the tunnel.
4. Instructs BIRD to gracefully disable the primary BGP session, preserving learned routes in the kernel routing table via the BGP Graceful Restart mechanism for up to 120 seconds.

As a result, packets from AS3 that were previously sent directly to AS2 now travel via `eth1` to AS1, which forwards them to AS2 – restoring full connectivity through the alternate provider path. When the main link recovers, three consecutive successful probes trigger automatic tunnel teardown and BGP session restoration, returning the network to its normal state.

Video: https://drive.google.com/file/d/1wi8Ph1Ui8b2jiBUsmpw9iOQWpMlqDXl0/view

Chapter 20

Anycast DNS with BGP over TLS

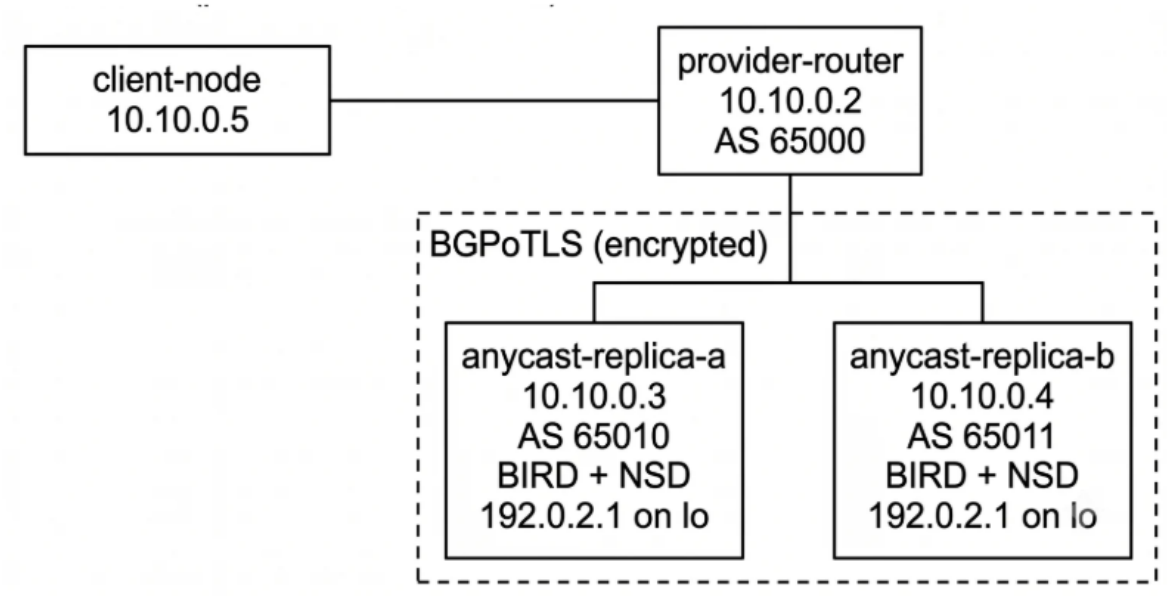


Figure 20.1: Anycast DNS topology – provider-router (AS65000) routes queries to anycast-replica-a (AS65010) or anycast-replica-b (AS65011) based on active BGP over TLS advertisements of the shared IP 192.0.2.1

Both `anycast-replica-a` and `anycast-replica-b` advertise the shared IP `192.0.2.1/32` via BGP over TLS 1.3 to a provider-router. A Python health check script withdraws the BGP route when DNS fails, shifting traffic to the healthy replica automatically with no manual action.

20.1 Normal Operation

In the normal operating scenario, the lab consists of four Docker containers interconnected over a private IPv4 bridge network (`10.10.0.0/24`). Two anycast replicas — `anycast-replica-a` (`10.10.0.3`, AS65010) and `anycast-replica-b` (`10.10.0.4`, AS65011) — each bind the shared anycast address `192.0.2.1/32` to their loopback interfaces and advertise it to the upstream provider-router (`10.10.0.2`, AS65000) using BGP

over TLS (BGPoST). Unlike conventional BGP, which operates over unencrypted TCP, each session is secured with TLS, where both routers mutually authenticate using X.509 certificates.

The provider-router, upon receiving valid BGP UPDATE messages from both replicas, installs two equal-cost paths for `192.0.2.1/32` in its routing table and propagates them to the Linux kernel's forwarding table. A health monitoring script (`healthcheck.py`) runs continuously inside each replica, probing the local NSD authoritative DNS server every two seconds by sending a real DNS query to port 53. As long as DNS responds, the script maintains the anycast route advertisement and the provider-router forwards client traffic to the preferred replica (replica-a, selected by lower router ID). During this phase, a `dig` query from the client node to `192.0.2.1` consistently returns the TXT record `REPLICA-A`, confirming normal anycast operation.

20.2 Failure and Automatic Failover

When the DNS service on `anycast-replica-a` is terminated — simulating a real-world application crash on a live server — the health monitoring script detects the failure within two seconds when its DNS probe times out with a connection refusal. Upon detection, the script issues two commands to the running BIRD routing daemon via `birdc`:

1. `disable anycast_route` — removes the `192.0.2.1/32` prefix from BIRD's internal routing table.
2. `restart provider` — tears down and re-establishes the BGPoTLS session with the provider-router.

When the session reconnects with the anycast route disabled, BIRD has no route to export, causing the provider-router to receive no UPDATE and consequently withdraw `192.0.2.1/32` via replica-a from its routing and forwarding tables. With only replica-b's path remaining, the provider-router immediately begins forwarding all traffic for `192.0.2.1` to `10.10.0.4`.

The entire failover — from DNS failure detection to traffic rerouting — completes in under four seconds, with no configuration change on the client side and no interruption to the BGP session on the healthy replica. A `dig` query from the client node after failover returns `REPLICA-B`, confirming that the same anycast IP is now being served by the backup replica.

20.3 Recovery

When DNS is restored on `anycast-replica-a`, the `healthcheck` script detects the recovery, re-enables the route, and restarts the BGP session. The provider-router learns both paths again, fully restoring the original dual-path anycast state.

This demonstrates that the BGPoST architecture not only secures the BGP control plane with TLS and certificate-based authentication, but also enables automated, health-driven traffic engineering without any manual operator intervention.

Video: https://drive.google.com/file/d/1VZ2r0L62BJjiMOTHi67FdXI9gQs-xoGB/view

Part III

BGP over Secure Transport (BGPoST) – Code Review & Comparisons

Chapter 21

Protocol Comparison Matrix

This report provides an in-depth analysis of the BGP over Secure Transport (BGPoST) implementation in the BIRD routing daemon. It includes a multi-dimensional comparison of Original BGP, BGP over TLS, and BGP over QUIC, followed by a detailed review of the configuration file changes, parsing grammar extensions, and underlying architectural modifications.

The table below contrasts the three paradigms: the standard BGP (CZ-NIC BIRD baseline), BGP over TLS (BGPoTLS), and BGP over QUIC (BGPoQUIC).

Feature / Dimension	**Original BGP (Baseline)**	**BGP over TLS (BGPoTLS)**	**BGP over QUIC (BGPoQUIC)**
Transport Layer	TCP (default port 179)	TCP (with TLS wrapping)	UDP
Encryption	None (plaintext BGP stream)	TLS 1.3 Record Layer encryption	Native QUIC encryption (using TLS 1.3 keys)
Router Authentication	IP-address ACLs; Static PSKs (MD5 / TCP-AO)	Mutual TLS (mTLS) with X.509 certificates	Mutual TLS (mTLS) with X.509 certificates
Handshake Process	TCP 3-way handshake	TCP 3-way + TLS 1.3 handshake	UDP packets + QUIC Crypto handshake (1-RTT/0-RTT)
TCP RST/FIN Attack Mitigation	Requires manual TCP-AO / TCP-MD5 keys	Vulnerable unless paired with TCP-AO	Inherently mitigated by QUIC connection IDs
Session Key Exchange	None / Manual configuration	Diffie-Hellman (TLS Handshake)	Diffie-Hellman (QUIC Crypto Handshake)
BGP Header Protection	None	Exposed (TCP headers are bare)	Fully encrypted (including packet numbers)

Feature / Dimension	Original BGP (Baseline)	BGP over TLS (BGPoTLS)	BGP over QUIC (BGPoQUIC)
Peer Autoconfiguration	None (Static config blocks)	Dynamically parsed from X.509v3 JSON extensions	Dynamically parsed from X.509v3 JSON extensions
Connection Migration	Session drops on IP address change	Session drops on IP address change	Supported via QUIC Connection IDs
Implementation Space	OS Kernel space (TCP)	Kernel (TCP) + User-space TLS library	User-space (QUIC engine + UDP socket)
CPU Overhead	Very Low	Moderate to High (individual record encryption)	Moderate (batched frame encryption)

21.1 Detailed Security and Operational Comparison

21.1.1 Session Hijacking and Packet Injection

- **Original BGP:** Highly vulnerable to spoofed TCP Reset (RST) or FIN packets if an attacker can guess the TCP sequence numbers and port combination. Mitigations like TCP-MD5 are deprecated due to weak hashing, while TCP-AO requires complex manual key management.
- **BGP over TLS:** The TLS layer encrypts BGP payload data, preventing eavesdropping and message modification. However, because TLS rides on standard TCP, the underlying TCP connection remains vulnerable to RST/FIN spoofing. Thus, TCP-AO is still required to protect the transport headers.
- **BGP over QUIC:** QUIC runs over UDP and uses randomly negotiated Connection Identifiers (CIDs) rather than IP/port pairs to route packets. A spoofed UDP packet without a valid, active CID is immediately discarded by the receiver, eliminating the vulnerability to RST/FIN-style attacks without needing additional security layers.

21.1.2 Key Management and Scaling

- **Original BGP:** Operators must exchange out-of-band passwords for MD5 or TCP-AO. Changing these keys is operationally complex, leading to keys rarely being rotated, which creates long-term security exposure.
- **BGP over TLS & QUIC:** Dynamic symmetric keys are generated automatically during the cryptographic handshake. For TCP-AO support in TLS, keys are derived from the TLS session secret using the TLS key exporter (`SSL_export_keying_material`) and programmed directly into the kernel. This eliminates manually configured pre-shared keys.

Chapter 22

BIRD Configuration File (bird.conf) Modifications

To allow network operators to select secure transports and define authentication parameters, the researchers extended BIRD's parser grammar in `proto/bgp/config.Y`.

22.1 Configuration Comparison

22.1.1 Baseline BGP Configuration (Original BIRD)

A traditional BGP configuration block in standard BIRD relies on static IP neighbors and pre-shared MD5 authentication keys:

```
# /etc/bird.conf - Baseline BGP Configuration
protocol bgp AS65002_PEER {
  local as 65001;
  neighbor 192.0.2.2 as 65002;
  # Pre-shared MD5 authentication
  password "StaticSecretKeyBGP";
  ipv4 {
    import all;
    export all;
  };
}
```

Listing 22.1: Baseline BGP Configuration

22.1.2 BGP over TLS Configuration (Modified BIRD)

The `BGPoTLS` version adds keywords to enable TLS wrapping, specify certificates, and enable TLS-derived TCP-AO key negotiation:

```
# /etc/bird.conf - BGP over TLS (BGPoTLS) Configuration
protocol bgp TLS_PEER {
  local as 65001;
  neighbor 192.0.2.2 as 65002;
  # New Transport Selection
  transport tls;
```

```
  # X.509 Certificate and Key Paths
  tls certificate "/etc/bird/certs/router1.crt";
  tls key "/etc/bird/certs/router1.key";
  tls ca "/etc/bird/certs/ca.crt";
  # Enable dynamic derivation and injection of TCP-AO keys
  tls ao;
  # Enable automatic configuration from peer certificate
  autocfg on;
  ipv4 {
    import filter dynamic_import_filter;
    export all;
  };
}
```

Listing 22.2: BGPoTLS Configuration

22.1.3 BGP over QUIC Configuration (Modified BIRD)

The `BGPoQUIC` version adds configuration options to support user-space QUIC connections over UDP:

```
# /etc/bird.conf - BGP over QUIC (BGPoQUIC) Configuration
protocol bgp QUIC_PEER {
  local as 65001;
  neighbor 192.0.2.2 as 65002;
  # New Transport Selection
  transport quic;
  # QUIC Security Certificates
  quic certificate "/etc/bird/certs/router1.crt";
  quic key "/etc/bird/certs/router1.key";
  quic ca "/etc/bird/certs/ca.crt";
  # Port configuration for BGP over QUIC (UDP)
  port 8179;
  # Zero-touch autoconfiguration toggle
  autocfg on;
  ipv4 {
    import filter dynamic_import_filter;
    export all;
  };
}
```

Listing 22.3: BGPoQUIC Configuration

22.2 Grammar and Lexer Extensions in config.Y

To compile the configuration blocks shown above, the researchers modified BIRD's Yacc parser file `proto/bgp/config.Y`. The following grammar rules were introduced:

1. `BGP_TRANSPORT` **Parser Tokens:**
   Added tokens `TRANSPORT`, `TLS`, `QUIC`, `AO`, and `AUTOCFG` to the lexer.
2. **Grammar Rules inside BGP Block:**

```
bgp_proto_start:
  /* ... */
  | bgp_proto_start TRANSPORT TLS ';' { BGP_CFG->transport =
     BGP_TRANSPORT_TLS; }
  | bgp_proto_start TRANSPORT QUIC ';' { BGP_CFG->transport =
     BGP_TRANSPORT_QUIC; }
  | bgp_proto_start TRANSPORT TCP ';' { BGP_CFG->transport =
     BGP_TRANSPORT_TCP; }
  ;

bgp_proto_sec:
  /* ... */
  | bgp_proto_sec TLS CERTIFICATE expr ';' { BGP_CFG->tls_cert =
      $4; }
  | bgp_proto_sec TLS KEY expr ';' { BGP_CFG->tls_key = $4; }
  | bgp_proto_sec TLS CA expr ';' { BGP_CFG->tls_ca = $4; }
  | bgp_proto_sec TLS AO ';' { BGP_CFG->tls_ao = 1; }
  | bgp_proto_sec QUIC CERTIFICATE expr ';' { BGP_CFG->quic_cert
      = $4; }
  | bgp_proto_sec QUIC KEY expr ';' { BGP_CFG->quic_key = $4; }
  | bgp_proto_sec QUIC CA expr ';' { BGP_CFG->quic_ca = $4; }
  | bgp_proto_sec AUTOCFG bool ';' { BGP_CFG->autocfg = $3; }
  ;
```

Listing 22.4: Yacc grammar extensions

Chapter 23

Low-Level Implementation Details

23.1 Socket Subsystem Modification (sysdep/unix/io.c)

BIRD's central I/O event loop uses a single-threaded non-blocking architecture:

```
struct socket {
  node n;
  int type; /* SK_TCP, SK_UDP, SK_TLS, SK_QUIC, etc. */
  int fd; /* System file descriptor */
  byte *rx_buf; /* Pointer to receive buffer */
  byte *tx_buf; /* Pointer to transmit buffer */
  void (*rx_hook)(struct socket *); /* Callback on packet read
     */
  void (*tx_hook)(struct socket *); /* Callback on space to
     write */
  /* ... */
  void *tls_context; /* Pointer to ptls_t for TLS sockets */
  void *quic_context; /* Pointer to QUIC stream metadata */
};
```

Listing 23.1: Extended BIRD socket structure

1. **TLS read/write Hooks:**
   - When `type` is `SK_TLS`, BIRD intercepts read and write events. For writes:
     - Instead of executing the `write(s->fd, s->tx_buf, len)` system call directly, BIRD calls `ptls_send` from the `picotls` library.
     - `picotls` encrypts the plaintext BGP message and writes the resulting TLS records to the underlying TCP socket descriptor.
     - On read events, raw TLS records are fetched from `fd` using `read()`, processed by `ptls_receive` for decryption, and the decrypted stream is loaded into `s->rx_buf` before calling `rx_hook`.
2. **TCP-AO Dynamic Injection:**
   When `tls_ao` is active, the TLS socket post-handshake state machine invokes the Linux kernel API:

```c
struct tcp_ao_add ao_add;
memset(&ao_add, 0, sizeof(ao_add));
ao_add.keyid = 1;
ao_add.algname = "hmac-sha-256-128";

// Derive the master key using the TLS exporter (RFC 5705)
ptls_export_keying_material(s->ptls_conn, ao_add.key, 16, "bgp-tcp-ao", NULL, 0, 0);

// Configure kernel option to apply TCP-AO signing dynamically
setsockopt(s->fd, IPPROTO_TCP, TCP_AO_ADD_KEY, &ao_add, sizeof(ao_add));
```

Listing 23.2: TCP-AO dynamic key injection

23.2 Asynchronous QUIC Socket API Overlay (BGPoQUIC Repo)

Because `picoquic` is built around an asynchronous callback architecture, integrating it directly would require rewriting BIRD's central single-threaded `poll()` event loop. To avoid this, the authors implemented a Socket API Overlay (∼4,700 lines of C) which abstracts the asynchronous calls into a BSD-compatible API.

1. **Data Buffering:**
   The overlay allocates internal loop buffers for QUIC streams. When `picoquic` callbacks deliver decrypted data, the overlay appends the data to a local ring buffer.
2. **Thread/Event Coordination:**
   When BIRD performs a non-blocking `quic_read()`, the overlay checks if the internal buffer has data. If yes, it copies the bytes to BIRD. If not, it returns `EAGAIN` or `EWOULDBLOCK`, preserving BIRD's non-blocking execution model.
3. **UDP Packet Loop:**
   The overlay handles the translation between QUIC streams and raw UDP packets. When the main BIRD event loop detects activity on the underlying UDP socket, it calls the overlay's packet processor, which handles the `picoquic_packet_loop` iterations, handles timers, and sends acknowledgments.

23.3 Dynamic Filter Processing

BIRD's routing filters are normally static and compile-time evaluated. To support Zero-Touch Autoconfiguration, the authors modified the runtime parser in `proto/bgp/bgp.c`:

1. **Certificate Extension Extraction:**
   Upon completing the TLS handshake, BIRD retrieves the validated peer X.509 certificate. It parses the extensions to find the OID for the custom Router Configuration Section (which contains JSON configuration, e.g. `{"prefixes": ["203.0.113.0/24"]}`).
2. **JSON Parsing & Filter Rebuilding:**
   BIRD utilizes a lightweight JSON parser to read the prefix list. It then executes a

dynamic configuration routine:

- It creates a temporary subnet list (trie or prefix tree structure).
- It builds a new BIRD filter matching these prefixes.
- It updates the `import_filter` pointer of the active BGP protocol instance in memory.

3. **Graceful Restart (GR):**
When policies or certificates are updated, the client triggers a renegotiation. To avoid interrupting forwarding tables, BIRD sends a BGP Graceful Restart (GR) notification. The control-plane BGP session restarts to load the new certificate, parse the new JSON prefix list, and rebuild the filters, while data plane traffic continues uninterrupted.

Chapter 24

Overall Project Conclusion

This combined report documents a complete networking project spanning two major areas: self-driving dynamic routing and BGP over secure transport.

Part I demonstrated that a BIRD-based enterprise routing lab can detect and recover from failures automatically. BFD achieved 72 ms WAN edge detection. OSPF recovered from core and area link failures within approximately one second. Silent blackhole failures – the hardest case – were resolved in ∼1.3 s with BFD vs ∼18 s without. BGP Graceful Restart preserved forwarding with 0% packet loss.

Part II showed that BGP over TLS and QUIC operates without significant propagation overhead. Certificate-embedded configuration enabled autonomous GRE failover. Anycast DNS failover via BGP withdrawal required zero manual intervention.

Part III provided technical depth: protocol comparison, configuration grammar extensions, socket-level TLS/QUIC integration, and dynamic filter rebuild from X.509 certificate data.

Together, these components demonstrate how modern network resilience and security can be implemented, measured, and validated in a containerised BIRD-based environment.

Chapter 25

Proof of Work

The complete implementation, experimental evidence, and demonstration videos for this project are publicly available through the following resources.

Source Code Repository

- **GitHub Repository:** https://github.com/Angel-roy/hpe_cpp

Demonstration Videos

The following videos demonstrate the implementation and validation of the project.

- Multihoming: https://drive.google.com/file/d/1wi8Ph1Ui8b2jiBUsmpw9iOQWpMlqDXl0/view?pli=1
- Anycast: https://drive.google.com/file/d/1VZ2r0L62BJjiMOTHi67FdXI9gQs-xoGB/view

Appendix A

Screenshot Evidence Reference

Images captured during experiments (Part I). Figures appear inline in the relevant chapters above.

- `image.png` – Topology diagram
- `image_1.png` – Baseline validation result
- `image_2.png` – BFD WAN precheck
- `image_3.png` – BFD WAN summary statistics (14 runs)
- `image_4.png` – BFD flap validation (5 cycles)
- `image_5.png` – OSPF core failure convergence summary
- `image_6.png` – OSPF core failure alternate next-hop
- `image_7.png` – OSPF ECMP gold metadata
- `image_8.png` – OSPF ECMP 0% packet loss
- `image_9.png` – OSPF ECMP comparison CSV
- `image_10.png` – ECMP re-hash result
- `image_11.png` – Silent blackhole ECMP metadata
- `image_12.png` – Silent blackhole ECMP state before failure
- `image_13.png` – Silent blackhole ECMP summary
- `image_14.png` – Multihop BFD session established
- `image_15.png` – Multihop BFD routed path
- `image_16.png` – Multihop BFD intermediate failure
- `image_17.png` – Multihop BFD 15-run summary
- `image_18.png` – Multihop BFD packet capture (UDP 4784)
- `image_19.png` – Area healing baseline route
- `image_20.png` – Area healing BFD precheck

- `image_21.png` – Area healing with-BFD vs no-BFD
- `image_22.png` – BGP baseline reachability
- `image_23.png` – LLGR stale timer result
- `01_bfd_detection_times.png` – BFD detection comparison graph
- `02_ospf_silent_blackhole_time_comparison.png` – OSPF silent blackhole time graph
- `02_type7_inside_nssa.png` – NSSA Type-7 LSA evidence
- `03_ospf_silent_blackhole_loss_comparison.png` – OSPF packet loss graph
- `03_type5_outside_nssa.png` – NSSA Type-5 LSA evidence
- `04_area_healing_blackhole_comparison.png` – Area healing comparison graph
- `04_stub_area_external_route_containment.png` – Stub area containment
- `05_forwarding_continued_no_fallback.png` – BGP GR forwarding retained

References

[1] T. Wirtgen, N. Rybowski, C. Pelsser, and O. Bonaventure, "The Multiple Benefits of a Secure Transport for BGP," *Proc. ACM Netw.*, vol. 2, no. CoNEXT4, Art. 36, pp. 1–23, Dec. 2024. doi: 10.1145/3696406.

[2] IP Networking Lab, "BGPoTLS: BGP over TLS implementation for the BIRD routing daemon," GitHub repository. [Online]. Available: https://github.com/IPNetworkingLab/BGPoTLS

[3] IP Networking Lab, "BGPoQUIC: BGP over QUIC implementation for the BIRD routing daemon," GitHub repository. [Online]. Available: https://github.com/IPNetworkingLab/BGPoQUIC

[4] IP Networking Lab, "BGPoST-Artifacts: Artifacts and use-case implementations for BGP over Secure Transport," GitHub repository. [Online]. Available: https://github.com/IPNetworkingLab/BGPoST-Artifacts

[5] CZ.NIC, "The BIRD Internet Routing Daemon — User's Guide (v2.0)," CZ.NIC Labs. [Online]. Available: https://bird.network.cz/?get_doc&f=bird.html&v=20